dbt Certification ��� Checkpoint 1: Build a Foundation

Checkpoint 1 — Build a Foundation (Meta Study Guide)

> Synthesized exam-prep guide for Checkpoint 1. Reorganizes the 29 per-resource cheat sheets by topic so concepts cluster and cross-reference cleanly.

The shape of Checkpoint 1

CP1 is **the dbt mental model**. Every other checkpoint assumes you have: - A worldview (the Viewpoint). - A project structure (How we structure). - Sources, models, materializations, and the run/build/test/seed/snapshot/docs commands. - Awareness of incremental models and the v1.10+ `--empty` / `--sample` flags.

Read this guide top-to-bottom. Each section links to the deeper per-resource sheets in this directory.

1. The dbt worldview

Read first: `01-dbt-viewpoint.md`

dbt was conceived to fix analytics workflows by **borrowing software engineering practices**: - Version control everything; code review every change. - Separate dev and prod environments with SLAs. - Modularity: a table's schema is its public interface; `ref()` is the implementation. - Tests, docs, deprecation processes — like a real software project. - Design for **maintainability** — most cost is in maintenance.

Every dbt feature (refs, sources, tests, docs, environments, contracts) traces back to a Viewpoint principle. When the exam asks *why* dbt does something, the answer usually lives here.

2. Project structure (the most-questioned topic)

Read in order: - `02-how-we-structure-overview.md` — the arc and three layers. - `03-how-we-structure-staging.md` — atoms. - `04-how-we-structure-intermediate.md` — molecules. - `05-how-we-structure-marts.md` — cells / entities. - `06-how-we-structure-rest.md` — YAML, auxiliary folders, project splitting.

The arc you must internalize

Source-conformed → business-conformed. Every other rule serves this trajectory.

Layer	Atoms / molecules / cells	Group by	Default materialization	Allowed transforms
staging	atoms	source system	view	rename, cast, basic compute, categorize. **No joins** (use `base/`), **no aggregations**
intermediate	molecules	business area	ephemeral (or view in custom schema)	re-grain, isolate complexity, structurally simplify joins
marts	cells	department/ domain	table → incremental	denormalize aggressively (unless on Semantic Layer)

Naming conventions

- Staging: `stg_[source]__[entity]s.sql` — **double underscore** between source and entity.
- Intermediate: `int_[entity]_[verb]s.sql` — verbs (`pivoted`, `summed`, `joined`).
- Marts: plain English by entity (`orders.sql`, `customers.sql`), no time-grain suffix.
- YAML: `_[directory]__models.yml`, `_[directory]__sources.yml`, `_[directory]__docs.md`.

The `dbt_project.yml` config cascade

```
models:
  jaffle_shop:
    staging: { +materialized: view }
    intermediate: { +materialized: ephemeral }
    marts:
      +materialized: table
      finance: { +schema: finance }
      marketing: { +schema: marketing }
```

Set defaults at the highest scope; override only where needed. Folders > tags. Tags mark exceptions.

Other folders

- `seeds/` — small lookup tables (NOT source data — use EL).
- `analyses/` — Jinja queries, version-controlled, not built into the warehouse.
- `tests/` — singular tests (cross-model logic).
- `snapshots/` — SCD2 capture.
- `macros/` — DRY helpers; document each with `_macros.yml`.

Project splitting

Use **dbt Mesh** when domains, governance (PII), or size demand it. Not for ML vs reporting — same source of truth.

3. Sources

Read: `08-sources.md`, `09-source-configs.md`.

A source is a named raw table you reference via `{{ source('src','tbl') }}` — only in staging models, 1:1 with source tables.

```
sources:
  - name: jaffle_shop
    database: raw
    config:
      freshness:               # v1.9+ inside config
        warn_after:  {count: 12, period: hour}
        error_after: {count: 24, period: hour}
      loaded_at_field: _etl_loaded_at   # v1.10+ inside config
    tables:
      - name: orders
```

Three big configs (v1.9+)

- `enabled` — toggle on/off (most common use: disabling sources from imported packages).
- `event_time` — column representing the actual event timestamp (powers microbatch and advanced CI).
- `meta` — free-form metadata.

Why sources exist

- Lineage (`{{ source() }}` puts them in the DAG).
- Source freshness monitoring (`dbt source freshness`).
- One place to edit if upstream changes schema/location.

Cross-references

- Source freshness mechanics → CP3 `cmd-source` cheat sheet.
- Source-driven builds → CP3 `state-selection` / `source_status:fresher+`.

4. Materializations

Read: `10-materializations.md`, `11-materializations-best-practices.md`.

Five built-ins

Materialization	Best for
view (default)	staging; cheap; always fresh
table	BI-queried marts; faster queries; full rebuild
incremental	event-style data; advanced; needs `unique_key`
ephemeral	DRY light transforms; inlined as CTE; **no contracts**, can't be ref'd from operations
materialized_view	warehouse-managed refresh (Snowflake uses Dynamic Tables instead)

The escalation ladder (memorize)

1. **View** → upgrade when **querying** is too slow.
2. **Table** → upgrade when **building** is too slow.
3. **Incremental** → only when full-refresh tables hurt.
4. (Optional sidestep) Materialized view if warehouse supports it.

Don't pre-optimize. Most models should never leave step 1 or 2.

Python models

Only `table` and `incremental`. No view or ephemeral. Models live in `.py` files; `model(dbt, session)` returns a DataFrame. Use `dbt.ref()`, `dbt.source()`, `dbt.config.get()`, `dbt.is_incremental`. Snowflake/BigQuery/Databricks only (and Fusion). See `22-python-models.md`.

5. Commands

Read each per-command sheet for syntax depth; here's the synthesis.

Production workhorses

- **`dbt build`** (`13-cmd-build.md`) — runs models, tests, snapshots, seeds (and v1.11+ UDFs) in DAG order. **Failing tests skip downstream**. Default for production.
- **`dbt run`** (`14-cmd-run.md`) — models only. `--full-refresh` forces incrementals to rebuild and makes `is_incremental()` return false everywhere.
- **`dbt test`** (`15-cmd-test.md`) — data + unit tests. Filter via `test_type:data` / `unit` / `generic` / `singular`. Standalone test reruns; use build for prod.

Build-time previews

- **`dbt show`** (`17-cmd-show.md`) — compile + run + preview rows. Single node at a time. `--limit n` pushes into SQL. `--inline "..."` for ad-hoc Jinja-SQL. Best for inspecting failing tests.
- **`dbt compile`** — render SQL into `target/` without executing. Use to inspect, debug, or hand-execute. (CP2 sheet.)

Data-type-specific

- **`dbt snapshot`** (`18-cmd-snapshot.md`) — runs Type-2 SCD snapshots. Usually run via `dbt build`.

- **`dbt seed`** (`19-cmd-seed.md`) — loads CSVs from `seeds/`. `--full-refresh` rebuilds; `--empty` (v1.12+) creates schema-only.
- **`dbt docs`** (`16-cmd-docs.md`) — `generate` builds docs site artifacts; `serve` hosts locally. Fusion replaces `generate` with `--write-catalog` on `run`/`build`/`compile`.

Two universal "save warehouse spend" flags

- **`--empty`** (`28-empty-flag.md`) — zero rows for all refs/sources; SQL still runs. CI/compile validation.
- **`--sample`** (`29-sample-flag.md`) — time-bounded slice of real data. Needs `event_time` on the sampled model and its parents. `.render()` opts a ref out.

---

6. `dbt_project.yml`

Read: `20-dbt-project-yml.md`.

Required, root-level. Naming gotcha: **dashes** inside `dbt_project.yml` for multi-word resource types (`saved-queries`, `semantic-models`); **underscores** everywhere else.

Key sections: - Identity: `name`, `version`, `profile`, `config-version: 2`. - Paths: `model-paths`, `seed-paths`, `snapshot-paths`, `macro-paths`, `analysis-paths`, `test-paths`, `docs-paths`, `asset-paths`, `function-paths`. - `require-dbt-version`, `query-comment`, `clean-targets`. - `flags:` block — global config defaults. - Resource-type config trees (`models:`, `seeds:`, etc.) — use `+` prefix to mark configs. - `on-run-start` / `on-run-end` hooks. - `dbt-cloud:` mapping (`project-id`, `defer-env-id`).

The `+` prefix marks a config (vs a folder-path key) inside resource trees.

---

7. Packages

Read: `21-packages.md`.

A package is a standalone dbt project consumed as a library. Reference its models with `ref('pkg_model')`. Sources from: **Hub** (recommended — handles transitive dedup), **git**, **tarball**, **local**, **private** (native via `private:` key with `provider:`).

Always pin versions (or use ranges) and commit `package-lock.yml`. `dbt deps` installs; `dbt clean` nukes; `dbt deps --upgrade` bumps pins.

Override package configs from your `dbt_project.yml` — your config wins.

---

8. Snapshots (Type-2 SCD)

Read: `24-snapshots.md` + `18-cmd-snapshot.md`.

Capture historical states of mutable source tables. Adds `dbt_valid_from`/`dbt_valid_to` columns.

Two strategies: - **`timestamp`** (recommended) — uses an `updated_at` column. Robust to schema change. - **`check`** — compares listed columns. Use when no reliable `updated_at`.

v1.9+ uses YAML config form. New configs: `dbt_valid_to_current` (set a sentinel like `9999-12-31` instead of NULL), `hard_deletes` (`ignore` | `invalidate` | `new_record`).

Keep snapshots in a **separate schema** with locked-down permissions; they cannot be rebuilt.

---

9. Incremental models

Read in order: - `25-incremental-models-overview.md` — concept. - `26-incremental-strategy.md` — `merge` / `append` / `delete+insert` / `insert_overwrite` / `microbatch`. - `27-incremental-microbatch.md` — time-series specialization.

Don't reach for incremental until you have to

View → table → incremental. Stop at the lowest tier that works.

The five strategies

Strategy	Use when
`merge`	You have a reliable `unique_key`; SCD1-like upsert
`append`	Append-only data; duplicates tolerable; cheapest
`delete+insert`	`merge` unsupported, or key isn't truly unique
`insert_overwrite`	Partition-aligned warehouses (BQ, Spark, Databricks, Snowflake); replace whole partitions
`microbatch`	Large time-series; backfill / late data; per-batch retry

Microbatch (the modern way for time-series)

Required configs: - `event_time` on the model AND on its direct parents you want filtered. - `begin` — initial-build start. - `batch_size` — `hour` / `day` / `month` / `year`. - Optional: `lookback`, `concurrent_batches`.

Each batch is idempotent. `dbt retry` reruns failed batches independently. Best practice: `full_refresh: false` so you don't accidentally rebuild from `begin`.

---

10. `grants` config

Read: `23-grants.md`.

Declare warehouse permissions in code. Applies to models, seeds, snapshots (not sources/tests).

- **Default = clobber**: a more-specific config replaces the less-specific list.
- **Add with `+select`**: prefix the privilege name with `+` to merge instead of replace.
- **Empty list `[]`** revokes all; **deleting the block** stops dbt managing grants entirely.
- Use **Jinja** for env-conditional grants: `{{ ['prod_role'] if target.name == 'prod' else ['dev_role'] }}`.

Fall back to **hooks** for row-level security, future grants, masking policies, anything not expressible via `grants`.

---

11. Best practice workflows

Read: `12-best-practice-workflows.md` and `07-refactoring-legacy-sql.md`.

Operational baselines (non-negotiable)

- Everything in version control with PR review.
- Separate dev/prod targets in `profiles.yml`.
- A documented style guide.
- `ref()` everywhere; sources for raw; rename/recast once.
- Test PK uniqueness + not-null at minimum.

Slim CI

```
dbt build --select state:modified+ --defer --state path/to/prod
```

Plus `--store-failures` for test debugging, `result:*` selectors for cheap reruns.

Refactoring legacy SQL — the 6-step process

1. Migrate 1:1 into a `models/` file. Run it.
2. Replace raw refs with **sources**.
3. Pick a strategy: **alongside** (recommended) or in-place.
4. Implement CTE groupings: import → logical → final → simple SELECT.
5. Port CTEs to staging/intermediate/marts models.
6. Audit with `audit_helper`.

---

12. The `--empty` and `--sample` flags

Read: `28-empty-flag.md`, `29-sample-flag.md`.

Flag	Input data	Use
`--empty`	0 rows	Cheap CI validation; schema-only builds
`--sample`	Time-bounded slice	Realistic dev with cheap reads
(none)	All rows	Production

Both flags ignore Python models. `--sample` requires `event_time` on the sampled models. Use `.render()` on a ref to opt it out of sampling.

Exam-ready summary tables

Materialization → use case

Materialization	Use case
view	Staging; light transforms
table	BI marts; many descendants
incremental	Slow-to-build tables, append-mostly
ephemeral	Light intermediate; one or two consumers
materialized_view	Managed-refresh alternative to incremental

Folder → group-by rule

Layer	Group by
staging	source system
intermediate	business area
marts	department/domain

Command → primary purpose

Command	Purpose
`dbt build`	Run + test + snapshot + seed (+ functions) in DAG order
`dbt run`	Models only
`dbt test`	Data + unit tests
`dbt snapshot`	T2 SCD capture
`dbt seed`	Load CSVs
`dbt show`	Preview SQL/test rows
`dbt docs generate`	Build docs site artifacts
`dbt docs serve`	Host docs site locally

Incremental strategy → typical adapter

Strategy	Typical adapters
`append`	postgres, redshift, snowflake, databricks
`merge`	most adapters
`delete+insert`	postgres, redshift, snowflake
`insert_overwrite`	bigquery, spark, databricks, snowflake
`microbatch`	most adapters in latest release

How to study from here

1. Read this meta guide top to bottom.
2. For each section, open the linked per-resource cheat sheets and skim the **Key takeaways**.
3. Make flashcards from the tables.

4. Build a tiny project that exercises: sources → staging view → intermediate ephemeral → marts table → snapshot → exposure → test. Use `dbt build`, then `dbt show` failing tests, then `dbt docs generate && dbt docs serve`.
5. Move to Checkpoint 2.

File index (this directory)

- `01-dbt-viewpoint.md`
- `02-how-we-structure-overview.md`
- `03-how-we-structure-staging.md`
- `04-how-we-structure-intermediate.md`
- `05-how-we-structure-marts.md`
- `06-how-we-structure-rest.md`
- `07-refactoring-legacy-sql.md`
- `08-sources.md`
- `09-source-configs.md`
- `10-materializations.md`
- `11-materializations-best-practices.md`
- `12-best-practice-workflows.md`
- `13-cmd-build.md`
- `14-cmd-run.md`
- `15-cmd-test.md`
- `16-cmd-docs.md`
- `17-cmd-show.md`
- `18-cmd-snapshot.md`
- `19-cmd-seed.md`
- `20-dbt-project-yml.md`
- `21-packages.md`
- `22-python-models.md`
- `23-grants.md`
- `24-snapshots.md`
- `25-incremental-models-overview.md`
- `26-incremental-strategy.md`
- `27-incremental-microbatch.md`
- `28-empty-flag.md`
- `29-sample-flag.md`

The dbt Viewpoint

Source: https://docs.getdbt.com/community/resources/viewpoint **Type:** Reading · Checkpoint 1

What it is

The 2016 manifesto that defines dbt's reason for existing: treating analytic code like software, so analytics teams can collaborate the way engineering teams do. It's foundational philosophy, not features — but exam questions often probe whether you can articulate *why* dbt looks the way it does.

The problem it names

Analytics teams ship slow, low-quality output because: - Knowledge is siloed; analysts duplicate work. - The same metric is calculated differently in different places. - Datasets are poorly understood by downstream consumers.

The fix isn't a new BI tool — it's a workflow borrowed from software engineering.

Three pillars

1. Analytics is collaborative

- **Version control** — all analytic code (SQL, Python, anything) belongs in git. Who changed what, when.
- **QA** — code that produces data or analyses must be reviewed and tested.
- **Documentation** — analyses are software; consumers need to know what "Revenue" means.
- **Modularity** — treat a table's schema as a public interface. No copy-paste of business logic; reference shared datasets so a definition change propagates once.

2. Analytic code is an asset

- **Environments** — analysts need a place to experiment without affecting production users; users need SLAs on prod data.
- **Service-level guarantees** — production analytics errors are bugs and should be treated with that urgency; retiring something from prod uses a deprecation process.
- **Design for maintainability** — most cost is in maintenance. Anticipate schema/data drift and write code that absorbs it.

3. Analytics workflows need automated tools

The manual plumbing (pulling source data, configuring environments, testing, deploying) should be automated. A canonical workflow: 1. Models/analyses pulled from version control, 2. Configured for the target environment, 3. Tested, 4. Deployed — all triggered by a single command.

dbt is dbt Labs' implementation of these ideas.

Why this matters for the exam

Several certification questions test *concepts* (separation of dev/prod, modularity via `ref`, the idea that the warehouse output is an "asset") more than commands. When you see questions about "why" — environments, why staging-models-as-views, why ref instead of hard-coded table names — the answer almost always traces back to a viewpoint principle.

Key takeaways

- dbt's design choices (refs, tests, docs, environments) map 1:1 to the pillars above.
- Modularity → schema is the public interface → `{{ ref('...') }}` is the implementation.
- Maintainability is the dominant cost — pick patterns that absorb change.
- Consistency across the team beats individual cleverness.

Related

- *How we structure our dbt projects* — operationalizes the modularity pillar.
- *Best practice workflows* — the workflow automation pillar in concrete form.

How We Structure Our dbt Projects — Overview

Source: https://docs.getdbt.com/best-practices/how-we-structure/1-guide-overview **Type:** Reading · Checkpoint 1 (also referenced in Checkpoint 3)

What it is

The opinionated dbt Labs guide to organizing a dbt project. The exam expects you to know this structure and the *why* behind it.

The core arc

Data should flow from **source-conformed → business-conformed**. Source-conformed = shape dictated by upstream systems you don't control. Business-conformed = shape dictated by your org's needs. Every layer in the project exists to move data along that arc.

Three primary `models/` layers

Layer	Analogy	Purpose	Default materialization
staging	atoms	One model per source table; clean, rename, cast	view
intermediate	molecules	Purpose-built transformation steps; isolate complexity	ephemeral or view in dev schema
marts	cells / proteins	Business entities; what end users query	table or incremental

Canonical project tree (Jaffle Shop)

```
jaffle_shop/
├── dbt_project.yml
├── packages.yml
├── analyses/
├── seeds/
├── macros/
├── snapshots/
├── tests/
└── models/
    ├── staging/
    │   ├── jaffle_shop/
    │   │   ├── _jaffle_shop__sources.yml
    │   │   ├── _jaffle_shop__models.yml
    │   │   ├── stg_jaffle_shop__customers.sql
    │   │   └── base/   # only when joins are needed pre-stage
    │   └── stripe/
    ├── intermediate/
    │   └── finance/
    │       └── int_payments_pivoted_to_orders.sql
    ├── marts/
    │   ├── finance/
    │   └── marketing/
    └── utilities/
```

Naming and folder rules

- **Staging**: group by *source system*, not by business domain or by loader.
- **Intermediate / marts**: group by *area of business concern* (finance, marketing).
- File names tell you everything: `stg_[source]__[entity]s.sql` (double underscore separates source and entity), `int_[entity]_[verb]s.sql`, marts named by the entity (`orders`, `customers`).
- Entities are plural (`orders`, not `order`) — reads like prose.

Why structure matters

- Reduces decision fatigue — your team spends bandwidth on hard problems, not "where does this file go."

- Folder = selector. `dbt build --select staging.stripe+` runs every downstream model from a single source.
- Folder = configuration boundary. `dbt_project.yml` cascades materializations and schemas based on directory.

Anti-patterns called out

- ❌ Staging subdirectories named after loaders (`fivetran/`, `stitch/`).
- ❌ Staging subdirectories named after business domains.
- ❌ Splitting concepts per team (`finance_orders` and `marketing_orders`) — that defeats single-source-of-truth.
- ❌ Over-optimizing too early — under ~10 marts, skip subdirectories.

Key takeaways

- Source-conformed → business-conformed is the only universal rule; everything else is convention to enforce it.
- Stay consistent and *document deviations* in your README.
- Folder structure is one of three knowledge-graph interfaces (DAG, folders, warehouse output) — keep all three coherent.

Related

- Staging / Intermediate / Marts / Rest-of-project — the four deeper guides this overview indexes.
- Best practice workflows; dbt_project.yml reference for the config-cascade mechanics.

How We Structure — Staging Layer

Source: https://docs.getdbt.com/best-practices/how-we-structure/2-staging **Type:** Reading · Checkpoint 1

What it is

The staging layer is the entry point: one model per source table, cleaning data into reusable atomic building blocks.

Folders

- Subdivide `models/staging/` **by source system** (`jaffle_shop/`, `stripe/`, `snowplow/`).
- ❌ Don't subdivide by *loader* (Fivetran, Stitch) — too broad.
- ❌ Don't subdivide by *business domain* — too early; you'll create competing definitions.

File-naming convention

- `stg_[source]__[entity]s.sql` — note the **double underscore** between source and entity. Disambiguates multi-word source names: `google_analytics__campaigns` reads clearly; `google_analytics_campaigns` is ambiguous.
- Plural entity names (`orders`, not `order`).
- One YAML config file per directory: `_jaffle_shop__models.yml`, `_jaffle_shop__sources.yml`. Leading underscore sorts them to the top.

What goes in a staging model

A standard staging model uses two CTEs: `source` (the only place that calls `{{ source(...) }}`) and `renamed` (transformations).

```
        -- stg_stripe__payments.sql
        with
        source as (
            select * from {{ source('stripe','payment') }}
        ),
        renamed as (
            select
                -- ids
                id as payment_id,
                orderid as order_id,
                -- strings
                paymentmethod as payment_method,
                status,
                -- numerics
                amount as amount_cents,
                amount / 100.0 as amount,
                -- booleans
                case when status = 'successful' then true else false end as
is_completed_payment,
                -- dates
                date_trunc('day', created) as created_date,
                -- timestamps
                created::timestamp_ltz as created_at
            from source
        )
        select * from renamed
```

Allowed transformations

✅ Renaming, type casting, basic computations (cents → dollars), categorizing (CASE expressions to bucket values).

Forbidden transformations

- ❌ **Joins** — except in `base` models (see below).
- ❌ **Aggregations** — grouping changes grain; you'll lose access to source detail downstream.

Materialization

- Default to **views** for the whole `staging/` directory via `dbt_project.yml`:

```
        models:
          jaffle_shop:
            staging:
              +materialized: view
```

- Views are cheap, always fresh, and not meant to be queried directly.

The 1-to-1 rule

- Staging models have a **1:1 relationship with source tables**.
- `source()` is used *only* in staging models. Downstream models use `ref()`.

Base models (the exception that proves the rule)

Use a `base/` subfolder when a join is genuinely needed *before* staging is meaningful: 1. **Joining a separate deletes table** — flag deleted records so all downstream models can filter them consistently. 2. **Unioning symmetrical sources** — e.g., per-region Shopify instances loaded as separate tables; union them in `base_` models, then write one `stg_` model on top.

Base models follow staging conventions but only do the non-joining transforms; the `stg_` model handles the joins/unions.

DRY principle

Push any "always want" transformation as far upstream as possible. If every downstream model needs `amount` as a float in dollars, do the cast in staging, not in 12 marts.

Productivity tip

Use the dbt-codegen package to auto-generate source YAML and staging model boilerplate. Write a few by hand to learn the style, then automate.

Key takeaways

- One model per source table, named `stg_[source]__[entity]s`.
- Group by source system in folders.
- Allowed: rename, cast, basic compute, categorize. Forbidden: joins (use `base/`), aggregations.
- Materialize as view.
- `source()` lives here and *only* here.

Related

- *Add sources to your DAG* — the `source()` macro itself.
- *Source configurations* — config keys you can apply to sources.

How We Structure — Intermediate Layer

Source: https://docs.getdbt.com/best-practices/how-we-structure/3-intermediate **Type:** Reading · Checkpoint 1

What it is

The middle layer that takes staging "atoms" and assembles them into purpose-built "molecules" on the way to marts. Intermediate models break complexity into named, testable steps.

Folders

- ✅ Subdivide `models/intermediate/` **by business area of concern** (`finance/`, `marketing/`).
- This is the layer where the project shifts from *source-conformed* to *business-conformed*.

File-naming convention

- `int_[entity]_[verb]s.sql` — emphasize **verbs**: `pivoted`, `aggregated_to_user`, `joined`, `fanned_out_by_quantity`, `funnel_created`.
- The double underscore from staging is **dropped** at this layer (we're past source distinctions).
- Exception: if a model still lives at source-system grain (e.g., `int_shopify__orders_summed` to be unioned later), keep the double underscore.
- Long descriptive names are worth it — anyone (even non-SQL readers) should grasp the model's job from the filename.

Example

```
        -- int_payments_pivoted_to_orders.sql
        {%- set payment_methods = ['bank_transfer','credit_card','coupon','gift_card'] 
-%}

        with
        payments as (
            select * from {{ ref('stg_stripe__payments') }}
        ),
        pivot_and_aggregate_payments_to_order_grain as (
            select
                order_id,
                {% for payment_method in payment_methods -%}
                    sum(case when payment_method = '{{ payment_method }}'
                        and status = 'success'
                        then amount else 0 end
                    ) as {{ payment_method }}_amount,
                {%- endfor %}
                sum(case when status = 'success' then amount end) as total_amount

            from payments
            group by 1
        )
        select * from pivot_and_aggregate_payments_to_order_grain
```

Note: CTE name (`pivot_and_aggregate_payments_to_order_grain`) tells the story for non-SQL readers; Jinja loop keeps the code DRY.

Materialization

- ❌ Not exposed to end users in the prod schema.
- ✅ **Ephemeral** — the simplest default; nothing materializes in the warehouse. Trade-off: harder to troubleshoot (the CTE is inlined into ref'ing models).
- ✅ **View in a separate custom schema with restricted permissions** — more robust for larger projects; you can inspect output without polluting prod.

Three canonical use cases

1. **Structural simplification** — instead of 10 joins in a mart, build two intermediate models with 5 joins each, then join those two in the mart. Improves readability, testability, and clarity.
2. **Re-graining** — fan out or roll up to the right grain before the mart. Example: fanning `orders` by `quantity` to create `order_items`.
3. **Isolating complex operations** — push tricky window functions or business logic into their own intermediate model so they can be debugged and tested in isolation.

DAG rule of thumb: "Narrow the DAG, widen the tables"

- Up through marts, the DAG should look like an arrowhead pointing right: many narrow staging models → fewer, wider intermediates → narrow set of marts.
- A model can have **many inputs**; multiple **outputs** (other models referencing this one) is a red flag — indicates the abstraction is wrong.

Don't over-engineer

The example uses subdirectories at this layer, but a small project (<10 marts) often doesn't need them. The goal is *single source of truth*, not maximum file count.

Key takeaways

- Verbs in the name (`pivoted`, `joined`, `summed`).
- Group by business domain, not source.
- Default to ephemeral; upgrade to view-in-custom-schema as the project grows.
- Three reasons to add an intermediate: simplify joins, fix grain, isolate complexity.
- Many inputs OK; many outputs from a single intermediate → re-think.

Related

- Materializations — for the ephemeral vs view trade-off.
- Marts guide — the consumer of intermediate models.

How We Structure — Marts Layer

Source: https://docs.getdbt.com/best-practices/how-we-structure/4-marts **Type:** Reading · Checkpoint 1

What it is

The marts layer is the *entity* or *concept* layer — wide, denormalized tables that represent the business entities end users actually query (`orders`, `customers`, `payments`).

Folders

- ✅ Group by **department or area of concern** (`finance/`, `marketing/`).
- Skip subfolders until you have ~10+ marts; don't over-optimize early.

File names

- ✅ Plain English by **entity** (`orders.sql`, `customers.sql`).
- ❌ Don't include time grain in filenames (`orders_per_day` is wrong — rollups belong in metrics, not in pure marts).
- ❌ Don't build the same concept differently per team (`finance_orders` vs `marketing_orders` is an anti-pattern). If finance genuinely needs a different concept, name it differently (`tax_revenue` vs `revenue`), not by department.

Example

```
        -- orders.sql
        with
        orders as (select * from {{ ref('stg_jaffle_shop__orders') }}),
        order_payments as (select * from {{ ref('int_payments_pivoted_to_orders') }}),

        orders_and_order_payments_joined as (
            select
                orders.order_id,
                orders.customer_id,
                orders.order_date,
                coalesce(order_payments.total_amount, 0) as amount,
                coalesce(order_payments.gift_card_amount, 0) as gift_card_amount

            from orders
            left join order_payments on orders.order_id = order_payments.order_id

        )
        select * from orders_and_order_payments_joined
```

Materialization

- ✅ **Tables** by default — speed matters for end-user queries.
- ✅ **Incremental** when a table grows large enough that full refresh is slow.
- Progression: start with view → upgrade to table when querying is slow → upgrade to incremental when *building* is slow.

Wide and denormalized

- Modern warehouses: storage is cheap, compute is expensive → denormalize aggressively.
- It's fine to bring `customer` data into the `orders` mart and `orders` data into the `customers` mart, even though that duplicates info — re-joining at query time is more expensive than storing redundant columns.

Joins: a soft cap

- 8 simple joins in a mart may be fine.
- 4 joins with complex window functions is often too much.
- Rule of thumb: if a mart joins **more than 4–5 concepts**, refactor into intermediate models. Two intermediates feeding a mart with two joins beats one mart with six.

Marts referencing marts

- Allowed and often correct (e.g., `customers` mart joins to `orders` mart to roll up lifetime value).
- Once data is materialized, re-using it is cheaper than recomputing from staging.
- Watch for: wasted recomputation, circular dependencies.

Marts are entity-grained

- Each mart represents a *single concept at a single grain*.
- The moment you start grouping multiple entities along a date spine (`user_orders_per_day`), you've left marts and entered **metrics** territory — that belongs in the Semantic Layer or a metric model.

Semantic Layer note

- Default guidance (no Semantic Layer): denormalize hard.
- If using the dbt Semantic Layer: keep marts *more normalized* — MetricFlow needs the flexibility. See *How we build our metrics* guide.

Troubleshooting tip

If errors seem to be raised in late models but actually originate upstream, temporarily materialize the upstream chain as tables so the warehouse throws the error at the real source.

Key takeaways

- One mart = one entity, named in plain English, plural.
- Materialize as table (or incremental for large/slow builds).
- Wide and denormalized — unless you're on the Semantic Layer.
- > 4 joins → consider refactoring into intermediates.
- Marts can reference marts; metrics live elsewhere.

Related

- Materializations — for the table/incremental progression.
- Intermediate layer — where to push complexity out of marts.
- Semantic Layer / How we build our metrics — when marts shouldn't be wide.

How We Structure — The Rest of the Project

Source: https://docs.getdbt.com/best-practices/how-we-structure/5-the-rest-of-the-project **Type:** Reading · Checkpoint 1

What it is

Everything outside `models/`: YAML configuration strategy, the auxiliary folders (`seeds`, `analyses`, `tests`, `snapshots`, `macros`), and when to split a monolithic project.

YAML strategy

✅ Config per folder (recommended)

- One `_[directory]__models.yml` per model directory (configures all models in that folder).
- For staging: also `_[directory]__sources.yml`.
- For doc blocks: `_[directory]__docs.md`.
- Leading underscore sorts YAML to the top of the folder, separating it from SQL files visually.
- File names don't need to be globally unique (unlike SQL model files).

❌ Config per project

Everything in one giant YAML file. Easy to find the file; impossible to find the config inside.

⚠️ Config per model

One YAML per SQL file. Some teams like it (instant file-tree spotting of models without configs). But the tab-juggling slows development for most teams.

✅ Cascade configs in `dbt_project.yml`

Set defaults at the directory level; override per-model only when needed.

```yaml
# dbt_project.yml
models:
  jaffle_shop:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      finance:
        +schema: finance
      marketing:
        +schema: marketing
```

Tags vs folders

- ✅ Folders are the *primary* selector mechanism (`dbt build --select marts.marketing`).
- ⚠️ Tags should mark **exceptions** — groups that cut across folders.
- ❌ Tagging every model defeats the purpose; over-reliance on tags is a structure smell.

Groups

A *group* is a collection of nodes in the DAG that enables intentional collaboration and **restricts access to private models**. Useful for multi-team or mesh setups (see `access` resource config).

The other folders

Folder	✅ Use for	❌ Avoid
`seeds/`	Small lookup tables (zip→state, UTM→campaign)	Loading source data — use an EL tool
`analyses/`	Auditing queries you want versioned with Jinja but never materialized; common use: `audit_helper` migration queries	–
`tests/`	Singular tests when generic packages (`dbt-utils`, `dbt-expectations`) don't cover the case; especially good for testing interactions between specific models	–
`snapshots/`	Type-2 SCDs from Type-1 source data	–
`macros/`	DRY-ing transformations; document each with `_macros.yml`	–

Project splitting (when to break up the monolith)

Default recommendation: use **dbt Mesh** rather than copying code between projects.

✅ Good reasons to split

- **Business domains owning their data products** (the primary Mesh use case).
- **Data governance** — e.g., PII restricted to a small team; import their staging project as a private package.
- **Project size** — thousands of models hurt developer experience.

❌ Bad reasons to split

- **ML vs. reporting** — same data, same single source of truth; both should consume the same marts and metrics.

Key takeaways

- One YAML per directory; sources YAML in staging dirs; doc blocks in `_*__docs.md`.
- Cascade defaults from `dbt_project.yml` down through folders.
- Folders select; tags label exceptions; groups govern access.
- Seeds = lookups, not source data.
- Singular tests are for cross-model logic that generic tests can't express.
- Split with Mesh when domains, governance, or size demand it — not for use-case taxonomy.

Related

- `dbt_project.yml` reference for full config cascade syntax.
- Groups / access configs for governance.
- Mesh module (Checkpoint 2) for project-splitting mechanics.

Refactoring Legacy SQL to dbt

Source: https://docs.getdbt.com/guides/refactoring-legacy-sql **Type:** Reading · Checkpoint 1

What it is

A six-step process for porting a giant legacy SQL script (typically a stored procedure) into modular dbt models without changing the output.

The six steps

1. Migrate 1:1 first

Paste the legacy SQL into a `.sql` file under `models/` and `dbt run` it. The goal: don't change logic, just get it executing in dbt. If you're also switching SQL dialects, syntax fixes are unavoidable — fix function-by-function, but resist "while I'm in here" rewrites. Every logic change creates auditing work.

2. Replace raw table references with sources

Define sources in YAML and swap `my_database.my_schema.my_table` for `{{ source('my_source', 'my_table') }}`. Three payoffs: - **Freshness reporting** (`dbt source freshness`). - **Dependency tracing** — see which migrated procs share raw tables, so you can consolidate base modeling. - **Habit formation** — config-driven references mean a source change is one YAML edit + one PR.

3. Choose a refactoring strategy

In-place: edit the migrated file directly. - ✅ No cleanup at the end. - ❌ More pressure to get it right; harder to audit; relies on git history to see what changed.

Alongside (recommended): copy the model into `/marts/` and refactor the copy. - ✅ End users keep referencing the old model while you work. - ✅ Audit by running old vs new side-by-side. - ✅ Smaller, incremental PRs. - ❌ Duplicate files until you deprecate.

4. Implement CTE groupings

Reorganize the model into a 4-part layout: 1. **Import CTEs** — one per source, `select * from {{ source(...) }}` (filters allowed). 2. **Logical CTEs** — pull subqueries out one at a time, named for the transformation they do. 3. **Final CTE** — the leftover top-level select, named clearly. 4. **Final select**: `select * from final`.

```sql
with
import_orders as (
    select * from {{ source('jaffle_shop','orders') }} where amount > 0

),
import_customers as (
    select * from {{ source('jaffle_shop','customers') }}
),
logical_cte_1 as ( -- math on import_orders ),
logical_cte_2 as ( -- math on import_customers ),
final_cte as ( -- join the two )
select * from final_cte
```

No nested subqueries. Anyone debugging can step through CTEs linearly.

5. Port CTEs to individual models

- **Import CTEs → staging models**, one per source table. Push always-needed transforms (renames, casts) here.
- **Logical CTEs → intermediate models** when complex or reusable. Keep simple ones as CTEs in the mart.
- **Final CTE → the mart**.

6. Audit the output

Use the `audit_helper` package to diff old vs new results. It generates the comparison queries for you — way faster than writing them by hand.

Key takeaways

- Migrate before refactoring; one logic change at a time = sane audits.
- Sources first — they pay dividends immediately.
- Prefer "alongside" refactoring over "in-place".
- 4-part CTE layout (import / logical / final / select) is the bridge from monolithic SQL to modular dbt.

- Don't skip the audit step — `audit_helper` exists for exactly this.

Related

- *How we structure our dbt projects* — where the refactored layers live.
- *Sources* — for the step-2 source declaration mechanics.
- *Best practice workflows* — the workflow this slots into.

Sources — Add Sources to Your DAG

Source: https://docs.getdbt.com/docs/build/sources **Type:** Documentation · Checkpoint 1

What it is

Sources name and describe the raw tables loaded by EL tools into your warehouse, so dbt can reference them via `{{ source() }}`, test them, and track their freshness.

What sources give you

- A single configurable mapping from logical source/table → physical database/schema/table.
- Lineage: `{{ source() }}` puts the source in the DAG.
- Source-level tests and documentation.
- **Source freshness** monitoring (SLA tracking).

Declaring a source (YAML)

```yaml
# models/staging/jaffle_shop/_jaffle_shop__sources.yml
sources:
  - name: jaffle_shop
    database: raw          # optional; defaults to target database
    schema: jaffle_shop    # optional; defaults to source `name`
    tables:
      - name: orders
      - name: customers
  - name: stripe
    tables:
      - name: payments
```

Selecting from a source

```sql
select ...
from {{ source('jaffle_shop', 'orders') }}
left join {{ source('jaffle_shop', 'customers') }} using (customer_id)
```

Compiles to `raw.jaffle_shop.orders`. Creates a DAG dependency between the model and the source.

> **Rule of thumb:** `{{ source() }}` should appear *only* in staging models. Everywhere else, use `{{ ref() }}`.

Testing & documenting sources

Sources accept the same `description`, `data_tests`, and `columns` keys as models:

```yaml
sources:
  - name: jaffle_shop
    description: "Replica of our app's Postgres DB"
    tables:
      - name: orders
        columns:
          - name: id
            description: Primary key
            data_tests:
              - unique
              - not_null
```

Source freshness

Configure

```yaml
sources:
  - name: jaffle_shop
    config:
      freshness:               # default for all tables in this source
        warn_after: {count: 12, period: hour}
        error_after: {count: 24, period: hour}
      loaded_at_field: _etl_loaded_at
    tables:
      - name: orders
        config:
          freshness:           # stricter override
            warn_after: {count: 6, period: hour}
            error_after: {count: 12, period: hour}
      - name: product_skus
        config:
          freshness: null      # opt out
```

- `loaded_at_field` is **required** unless dbt can use warehouse metadata.
- At least one of `warn_after` / `error_after` must be set, else freshness isn't computed.
- Source-level config cascades to all tables.
- (v1.9+) `freshness` and `loaded_at_field` moved under `config:` block.

Run it

```
dbt source freshness
```

Under the hood dbt runs:

```sql
select max(_etl_loaded_at) as max_loaded_at,
       convert_timezone('UTC', current_timestamp()) as calculated_at
from raw.jaffle_shop.orders
```

Filter (avoid full scans)

For very large tables:

```yaml
config:
  freshness:
    filter: "_etl_loaded_at >= date_sub(current_date(), interval 1 day)"
```

State-aware orchestration (Fusion)

The Fusion engine tracks freshness automatically via warehouse metadata — you only need explicit freshness config for SLA alerts, custom freshness logic, or source views.

Build-on-freshness pattern

```
dbt source freshness
dbt build --select source_status:fresher+ --state path/to/prod
```

Only rebuild models downstream of sources whose data has actually changed. Pairs well with a 30-min freshness snapshot cadence and an hourly rebuild job.

Key takeaways

- Sources = named raw tables; declared in YAML; selected via `{{ source('src', 'tbl') }}`.
- Use sources only in staging models; one staging model per source table.
- Test and document them like you do models.
- Freshness needs `loaded_at_field` and at least one threshold; configs cascade source → tables.
- `source_status:fresher+` enables freshness-driven incremental builds.

Related

- Source configurations (the configs themselves).
- `dbt source freshness` command.
- `loaded_at_field`, `event_time` resource properties.

Source Configurations

Source: https://docs.getdbt.com/reference/source-configs **Type:** Documentation · Checkpoint 1

What it is

The configuration keys you can apply to sources (in `dbt_project.yml` or in a source's YAML `config:` block). The big three: `enabled`, `event_time`, `meta`. Plus inherited general configs and the `freshness` config that lives at the property layer.

Where you can set source configs

In `dbt_project.yml` (under `sources:`)

```yaml
sources:
  <project-or-package-name>:
    <source-name>:
      <table-name>:          # optional, deepest level
        +enabled: true | false
        +event_time: my_time_field
        +freshness:
          warn_after:
            count: <int>
            period: minute | hour | day
        +meta:
          key: value
```

Inline in a properties YAML (`models/.../*.yml`)

```yaml
sources:
  - name: my_source
    config:
      enabled: true
      event_time: my_time_field
      meta: {owner: "data-platform"}
      freshness:
        warn_after: {count: 4, period: hour}
    tables:
      - name: my_table
        config:
          enabled: false
```

The keys (v1.9+)

Key	Purpose
`enabled`	Toggle a source or table on/off
`event_time`	Column representing the *real* timestamp of an event (not load time). Required for microbatch incremental; used for CI-vs-prod comparisons
`meta`	Free-form metadata (owners, source system tags, descriptions)
`freshness`	Move-to-config-block version of the freshness threshold (was a property pre-1.9)
`loaded_at_field`	(1.10+) column used to compute freshness; under `config:` block

Patterns

Disable all sources from a package

```yaml
# dbt_project.yml
sources:
  events:
    +enabled: false
```

Disable a specific source from a package

```yaml
sources:
  events:
    clickstream:
      +enabled: false
```

Disable a specific table

```yaml
sources:
  events:
    clickstream:
      pageviews:
        +enabled: false
```

Conditionally enable via a variable

```yaml
sources:
  - name: my_source
    tables:
      - name: my_source_table
        config:
          enabled: "{{ var('my_source_table_enabled', false) }}"
```

Source in a subdirectory (disabling via project YAML)

You must mirror the full path:

```yaml
sources:
  your_project_name:
    subdirectory_name:
      source_name:
        source_table_name:
          +enabled: false
```

Attach event_time for microbatch

```yaml
sources:
  events:
    clickstream:
      +event_time: event_timestamp
```

Required for `incremental_strategy='microbatch'`; also unlocks event-window comparison in advanced CI.

Attach meta

```yaml
sources:
  events:
    clickstream:
      +meta:
        source_system: "Google Analytics"
        data_owner: "marketing_team"
```

Freshness (project-wide default)

```yaml
sources:
  <resource-path>:
    +freshness:
      warn_after: {count: 4, period: hour}
```

Inheritance & resource paths

- Configs cascade by resource path: `<project>:<source>:<table>` (most-specific wins).
- A `+` prefix marks a config (vs a property name); paths without `+` are part of the resource path.
- Source-level config applies to all tables unless a table overrides.

Most common use case

Disabling sources from imported **packages** so you don't pollute your DAG or run freshness checks on tables you don't own.

Key takeaways

- Three primary keys (1.9+): `enabled`, `event_time`, `meta` — plus `freshness` and `loaded_at_field` in config block.
- Configure in `dbt_project.yml` for project-wide patterns; configure inline for source-specific overrides.
- Resource paths follow `<project>:<source>:<table>` depth.
- `event_time` is the doorway to microbatch and advanced CI comparisons.
- Disabling is the most common real-world use, especially for package sources.

Related

- Sources doc — the declaration mechanics.
- `event_time` resource config — full reference.
- Incremental microbatch — biggest consumer of `event_time`.

Materializations

Source: https://docs.getdbt.com/docs/build/materializations **Type:** Documentation · Checkpoint 1

What it is

Materializations are the *strategy* dbt uses to persist a model in the warehouse. Five built-ins plus custom materializations.

Materialization	What it is	Best for
view (default)	`create view as ...` each run	Cheap transforms; staging
table	`create table as ...` each run	BI-queried marts; reused downstream
incremental	Inserts/updates new rows since last run	Big event-style data
ephemeral	Inlined as a CTE into refs; not materialized	Light DRY logic early in DAG
materialized_view	Native warehouse MV (where supported)	Like incremental but warehouse refreshes it

Default: **view**.

How to configure

In `dbt_project.yml` (cascades by folder):

```yaml
models:
  my_project:
    events:
      +materialized: table
    csvs:
      +materialized: view
```

In the model SQL:

```sql
{{ config(materialized='table', sort='timestamp', dist='user_id') }}
select * from ...
```

In `properties.yml`:

```yaml
models:
  - name: events
    config:
      materialized: table
```

The five built-ins in depth

View

- `create view as`. No storage.
- ✅ Always fresh, cheap.
- ❌ Slow to query for heavy transforms or stacked views.
- 💡 Default. Use for staging and anything not heavy.

Table

- `create table as`. Stored.
- ✅ Fast to query.
- ❌ Rebuilds fully each run (can be slow); no auto-pickup of new source rows.
- 💡 Use for BI-queried marts and slow-built models reused downstream.

Incremental

- Adds/updates rows since the last run.
- ✅ Build time scales with new data, not all data.
- ❌ Advanced — needs `unique_key`, `incremental_strategy`, etc.
- 💡 Use only when full-refresh tables get too slow. Best for event-style/append-mostly data.

Ephemeral

- Inlined into ref-ing models as a CTE (prefixed `__dbt__cte__`).
- ✅ DRY, no warehouse clutter.
- ❌ Cannot `select` directly. Operations (e.g. `dbt run-operation`) can't `ref()` it. Harder to debug. **Does not support model contracts.**
- 💡 Use sparingly: light transforms early in DAG, referenced by one or two models.

Materialized View

- Native warehouse MV. Combines view ergonomics with table speed.
- ✅ Most platforms auto-refresh; `dbt run` becomes deploy-only (like a view) — no need to schedule refreshes.
- ❌ Fewer per-platform config knobs; not supported everywhere.
- 💡 Use when incremental would work but you'd prefer the platform manages the refresh.
- Uses `on_configuration_change` config to decide whether to alter or recreate when a setting changes.

> Note: `dbt-snowflake` doesn't support materialized views — it uses **Dynamic Tables** instead.

Python models — materialization limits

- ✅ `table`, `incremental` only.
- ❌ Cannot be `view` or `ephemeral`.
- Incremental Python models support the same `incremental_strategy` options as SQL (adapter-dependent).
- BigQuery/Dataproc: incremental Python supports `merge` but **not** `insert_overwrite`.

Decision flow (the canonical progression)

1. **Start with view.**
2. When the view is too slow to *query* → **table**.
3. When the table is too slow to *build* → **incremental**.
4. If the warehouse supports it and you want managed refresh → **materialized view**.

Custom materializations

You can define your own via macros — covered in *Creating new materializations*.

Key takeaways

- Default is view; only escalate when there's a real problem.
- Pick based on (a) query speed needed, (b) build time tolerable, (c) whether you want managed refresh.
- Ephemeral has real limitations: no direct select, no `ref` from operations, no contracts.
- Python = table or incremental, never view/ephemeral.
- Configure at the directory level via `dbt_project.yml` whenever possible.

Related

- Materializations best practices — when to escalate.
- Incremental models overview / strategy / microbatch.
- Snapshots — for SCD Type-2 (separate doc).

Materializations — Best Practices

Source: https://docs.getdbt.com/best-practices/materializations/1-guide-overview
Type: Documentation · Checkpoint 1

What it is

The opinionated guide on *when* to use which materialization, building on the materializations reference. Tldr: start simple, escalate only on real pain.

Why this matters

Materializations abstract the DDL/DML you'd normally write by hand. You declare *what* you want (table, view, incremental); dbt figures out *how* to build it for your warehouse. Choosing the right one is the difference between fast/cheap pipelines and slow/expensive ones.

Learning goals

- What materializations are
- How **table**, **view**, **incremental** differ
- When and where to apply each
- How to configure at multiple scopes (single model, folder, whole project)

The guiding principle: start as simple as possible

A three-tier escalation. Only move up when you hit a real problem.

Tier	Materialization	Move up when
🔍 1	**View**	The view is too slow to *query* for end users
⚒️ 2	**Table**	The table is too slow to *build* in your dbt jobs
📚 3	**Incremental**	(build time is the bottleneck) layer data in chunks

Materialized views are an optional sidestep at tier 2/3 if your warehouse supports them and you'd rather have the platform manage refreshes.

How this maps to project structure

Layer	Default materialization
staging	view
intermediate	ephemeral (or view in dev schema)
marts	table (escalate to incremental when build is slow)

Configuration scopes (most specific wins)

1. **Project-wide** in `dbt_project.yml`:

```yaml
models:
  my_project:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

2. **Per directory** — same mechanism, nested deeper.
3. **Per model** in the SQL via `{{ config(materialized='incremental') }}`.
4. **In `properties.yml`** for the model.

Use the cascade. Configure exceptions only at the level where they apply.

Common mistakes

- Making everything a table "to be safe" — wastes warehouse build time and money.
- Jumping straight to incremental on a 100k-row model — the complexity tax isn't worth it.
- Using ephemeral as the default for intermediate models on big projects — debugging becomes painful as model count grows; views in a custom schema are often better.

Prerequisites this guide expects you've done

- The Quickstart project.
- Familiar with `ref()`, models, and runs.
- (Optional) Read *How we structure our dbt projects* so the staging/intermediate/marts conventions click.

Key takeaways

- The three-tier escalation (view → table → incremental) is the canonical decision tree.
- Stop at the lowest tier that works; don't optimize speculatively.
- Set materialization at the highest scope that makes sense (folder), override only when needed.
- Materialized views are a managed-refresh alternative to incremental on supported warehouses.

Related

- Materializations (reference) — all five types in depth.
- Incremental models overview / strategy / microbatch — the next escalation step.
- How we structure our dbt projects — the layer-by-layer default materializations.

Best Practice Workflows

Source: https://docs.getdbt.com/best-practices/best-practice-workflows **Type:** Documentation · Checkpoint 1

What it is

The collective dbt-Labs wisdom on how to *operate* dbt day-to-day: version control, environments, project conventions, and CI workflows that scale.

Workflow practices

Version control everything

- All dbt projects in git; PR review required before merging to `main`.
- Feature branches for new work and bug fixes.

Use separate dev and prod environments

- `dev` target for local CLI work; `prod` target only for scheduled deployments.
- Configured via profile targets — see "Managing environments."

Adopt a style guide

- Codify SQL style, field naming, file conventions.
- dbt Labs' public style guide is a good starting point.

Project practices

Always `ref()`

- `{{ ref('model') }}` lets dbt infer dependency order *and* environment-correctness.
- Never select directly from `my_schema.my_table` for models.

Limit raw-data references — use sources

- Define raw data as sources (one place to update when schemas drift).
- No direct relation references in dbt Labs' own projects.

Rename & recast once (at the staging layer)

- Staging models select from a single source, do all renames and type casts, then everything downstream inherits.
- Eliminates duplicated transformation logic.

Break complex models up

Split when: - A CTE is duplicated across two models (→ extract a model). - A CTE changes grain (→ extract, then test it). - The SQL is too long to hold in your head.

Group models in directories

Use nested subdirectories — they enable: - Folder-level config in `dbt_project.yml`. - Folder-based selection (`--select marts.marketing`). - Convention enforcement (e.g., "marts can only ref marts or staging").

Add tests

At a minimum: every model has a primary key tested for `unique` and `not_null`.

Design the warehouse's information architecture

- Use custom schemas to group related models.
- Use prefixes (`stg_`, `fct_`, `dim_`) so SQL-client users can find query-ready tables.

Choose materializations wisely

Pattern: - **views** by default - **ephemeral** for lightweight not-end-user transforms - **tables** for BI-queried models and models with many descendants - **incremental** only when table build time becomes intolerable

Pro tips

Model selection when developing locally

Run just what you're touching: `dbt run --select my_model+` (the `+` includes downstream).

Slim CI — only modified models

```
dbt run  -s state:modified+ --defer --state path/to/prod/artifacts
dbt test -s state:modified+ --defer --state path/to/prod/artifacts
```

Reruns only models that changed (+ their descendants), with `--defer` reading unchanged parents from production. Saves time and money in CI.

Result-status selectors

Combine `state:modified+` with `result:error+`, `result:fail+`, etc.:

```
dbt build --select state:modified+ result:error+ --defer --state path/to/prod/artifacts
```

Rerun failures plus modifications in one command.

> Gotcha: `dbt test` overwrites `run_results.json`, so combining `result:error` + `result:fail` only works in a single `dbt build` invocation, not across sequential `dbt run` then `dbt test`.

Source-freshness-driven builds

```
dbt source freshness
dbt build --select source_status:fresher+ --state path/to/prod/artifacts
```

Only build downstream of sources that actually got new data.

Limit dev data via target

```
select * from event_tracking.events
{% if target.name == 'dev' %}
where created_at >= dateadd('day', -3, current_date)
{% endif %}
```

Or use the env var `DBT_CLOUD_INVOCATION_CONTEXT` (`prod`, `dev`, `staging`, `ci`).

Use `grants` resource config

Codify warehouse permissions in dbt; no more manual `GRANT` statements drifting from version control.

Separate source-centric from business-centric logic

Two passes: 1. Source-centric: union, dedup, re-alias, re-cast → staging. 2. Business-centric: define entities and metrics → marts. This is the structural reason for the staging/marts split.

Manage Jinja whitespace

Compiled SQL in `target/compiled/` showing weird whitespace? See Jinja's whitespace-control docs.

Key takeaways

- Version control, dev/prod separation, style guide — non-negotiable baselines.
- `ref()` everywhere; sources for raw; rename/recast once.
- Folder structure drives config, selection, and convention.
- Slim CI (`state:modified+ --defer`) is the highest-ROI workflow pattern.
- Match materializations to the speed/build-time trade-off, not to "feels right."

Related

- *How we structure our dbt projects* — the folder structure this references.
- Slim CI / `--defer` / state selection.
- `grants` resource config; environment variables.

`dbt build`

Source: https://docs.getdbt.com/reference/commands/build **Type:** Documentation · Checkpoint 1

What it does

Runs **models, tests, snapshots, seeds** (and, in 1.11+/Fusion, **functions**) in DAG order — all in one command. The canonical production deploy command.

When to use

Almost always, in production. `build` replaces the older pattern of `seed && run && snapshot && test` and adds the critical safety property: a failing test on a model **skips downstream models**.

Output artifacts

Writes one `manifest.json` and one combined `run_results.json` covering everything that was selected.

Skipping behavior

- A failing **test** on a model **skips** that model's downstream resources.
- Want a test to warn instead of block? Set `severity: warn`.
- Test with multiple parents (e.g. a `relationships` test): blocks/skips children of the most-downstream parent only.
- (v1.12+) A failing **model** also skips its downstream models — but you can override with `on_error: continue` on a model to let descendants run anyway.

Selection

- Standard syntax: `--select`, `--exclude`, `--selector` (the last dropped in 1.12+).
- `--resource-type` final filter: `dbt build --select "resource_type:function"`.
- **Tests use indirect selection**: `dbt build -s model_a` runs `model_a` *and* its tests (so long as those tests don't depend on unselected parents).
- Filter by test type: `--select test_type:unit` or `--select test_type:data`.

The `--empty` flag

Schema-only dry run: refs/sources are limited to zero rows. dbt still executes the SQL against the warehouse but avoids expensive reads. Validates that models *would* build without paying for the data. Pairs with seeds (`dbt seed --empty`) for CI.

Unit tests + data tests in one run

When unit tests are defined, `dbt build` processes: 1. Unit tests on the SQL model. 2. Materializes the model (only if unit tests pass). 3. Data tests on the materialized model.

Net effect: warehouse spend is skipped when unit tests catch bugs first.

Shared flags

`build` inherits all flags from `run`, `test`, `snapshot`, `seed`. Shared flags like `--full-refresh` apply across resource types — both models and seeds get full-refreshed.

Functions (v1.11+ / Fusion)

```
        dbt build --select "resource_type:function"
```

Builds user-defined functions (UDFs) as part of the DAG.

Example output

```
1 of 7 OK loaded seed file dbt_jcohen.my_seed           [INSERT 2 in 0.09s]
2 of 7 OK created view model dbt_jcohen.my_model        [CREATE VIEW in 0.12s]
3 of 7 PASS not_null_my_seed_id                         [PASS in 0.05s]
...
Done. PASS=7 WARN=0 ERROR=0 SKIP=0 TOTAL=7
```

Common commands

```
        dbt build                                               # everything
        dbt build --select staging.stripe+                      # downstream of a folder

        dbt build --select state:modified+ --defer --state path/to/prod
        dbt build --select source_status:fresher+ --state path/to/prod
        dbt build --select test_type:unit                       # unit tests only
        dbt build --empty                                       # schema dry run
```

Key takeaways

- `build` = run + test + snapshot + seed + (functions) in DAG order.
- Failing tests on parents skip downstream children — this is what makes `build` safer than separate commands.
- Tests are indirectly selected when you select the model.
- `--empty` is the cheap CI validation flag.
- v1.12+ adds model-level `on_error: continue` to opt out of downstream skipping.

Related

- `dbt run`, `dbt test`, `dbt seed`, `dbt snapshot` — the individual operations.
- Slim CI pattern: `state:modified+ --defer --state path`.
- `severity` config — control whether tests warn or error.

`dbt run`

Source: https://docs.getdbt.com/reference/commands/run **Type:** Documentation · Checkpoint 1

What it does

Compiles and executes **models only** against the target database. Does NOT run tests, snapshots, or seeds.

If you want everything in one go, use `dbt build`.

How it works

1. Compiles each model's SQL.
2. Resolves dependencies into a DAG.
3. Materializes models in dependency order, multi-threaded.
4. For replace-style materializations (table), dbt builds with a temporary name, then drops and renames in a single transaction (on adapters that support transactions). This minimizes downtime.

The `--full-refresh` flag

Forces incremental models to be rebuilt as full tables.

```
        dbt run --full-refresh
        dbt run -f                      # short form
```

Use when: - An incremental model's schema changed and needs to be recreated. - The model's logic changed and you need to reprocess history.

Behavior inside compiled models: - `flags.FULL_REFRESH` is `true`. - `is_incremental()` returns `false` for **all** models — so the incremental branch is skipped.

```
        select * from all_events
        {% if is_incremental() %}
          where collector_tstamp > (select coalesce(max(max_tstamp), '0001-01-01') fr
om {{ this }})
        {% endif %}
```

Selecting models

Use node selection syntax to limit what runs:

```
        dbt run --select my_model           # one model
        dbt run --select my_model+          # model + downstream
        dbt run --select +my_model          # upstream + model
        dbt run --select staging.stripe+    # folder + downstream
        dbt run --exclude my_model
```

The `--empty` flag

Schema-only dry run: refs/sources limited to zero rows. SQL still executes but won't scan input data. Validates compilation and dependency wiring.

Related global configs

- **Treat warnings as errors** — see `warn_error` / `warn_error_options`.
- **Failing fast** — `--fail-fast` to abort on first error.
- **Colorized logs** — `print_color` global config.

Status codes (dbt Cloud API)

When polling `list_runs`: | Code | Meaning | |—|—| | 1 | Queued | | 2 | Starting | | 3 | Running | | 10 | Success | | 20 | Error | | 30 | Canceled | | 40 | Skipped |

When to choose `run` over `build`

- You're iterating on model logic and don't want to wait for tests/seeds/snapshots.
- You have a workflow that handles tests as a separate concern.
- You only care about materializing models (e.g., a one-off backfill).

Common patterns

```
        dbt run                                             # all models
        dbt run --select my_model+                          # this and downstream

        dbt run --full-refresh --select my_incremental      # rebuild incremental

        dbt run --select state:modified+ --defer --state path/to/prod    # slim

        dbt run --empty                                     # compile-and-dry-run

        dbt run --fail-fast                                 # stop on first error
```

Key takeaways

- Models only — for everything use `build`.
- `--full-refresh` makes `is_incremental()` return false everywhere.
- Replace-style materializations swap atomically via a temp name + rename.
- `--empty` is your "does this compile and wire up?" check.
- Selection syntax (`+`, folder-path, `state:modified+`) is the productivity multiplier.

Related

- `dbt build` — superset.
- Node selection syntax / graph operators.
- Incremental models — the audience for `--full-refresh`.

`dbt test`

Source: https://docs.getdbt.com/reference/commands/test **Type:** Documentation · Checkpoint 1 (also referenced in Checkpoint 3)

What it does

Runs **data tests** (on models, sources, snapshots, seeds) and **unit tests** (on SQL models). Expects resources already exist — i.e. you should `dbt run`/`dbt seed`/`dbt snapshot` (or `dbt build`) first.

Quick reference

```
        # all tests (data + unit)
        dbt test

        # only data tests
        dbt test --select test_type:data

        # only unit tests
        dbt test --select test_type:unit

        # tests for a specific model (indirect selection)
        dbt test --select "one_specific_model"

        # tests across a whole package
        dbt test --select "some_package.*"

        # only singular tests
        dbt test --select "test_type:singular"

        # only generic tests
        dbt test --select "test_type:generic"

        # combine selectors with commas (intersection)
        dbt test --select "one_specific_model,test_type:data"
        dbt test --select "one_specific_model,test_type:unit"
```

Test type taxonomy

- **Data tests**
  - **Generic** — reusable, parameterizable, declared in YAML (`unique`, `not_null`, package tests, custom ones).
  - **Singular** — a single SQL file in `tests/` that returns failing rows.
- **Unit tests** — declared in YAML; test SQL transformation logic against synthetic input rows. Run separately from data tests (and run *before* materialization when invoked via `dbt build`).

Indirect selection

`dbt test --select my_model` pulls in: - Tests defined directly on `my_model`'s columns/properties. - Generic tests that reference `my_model`. - Excludes tests whose other parents aren't selected.

This is the same indirect-selection rule used by `dbt build`.

How tests work under the hood

- A test compiles into a SQL query that returns failing rows.
- 0 rows → pass. >0 → fail (or warn, depending on `severity`).
- Thresholds: `warn_if`, `error_if`, `severity` give fine-grained control (see data test configs).

Common patterns

```
        # CI: only modified tests
        dbt test --select state:modified --defer --state path/to/prod

        # Rerun failed tests
        dbt test --select result:fail

        # Test smarter — exclude known-failing test
        dbt test --select result:fail --exclude my_flaky_test --defer --state path/to/
prod

        # Test a source
        dbt test --select source:jaffle_shop
```

Test result statuses

- `pass` — 0 failing rows
- `fail` / `error` — >0 failing rows above the error threshold
- `warn` — >0 failing rows, but below error threshold or severity is warn
- `skipped` — depending on upstream failures (when run via build)

When to use `test` vs `build`

- `dbt test` alone is for: developing new tests, running tests after changing test logic, CI test-only jobs.
- `dbt build` is preferred in production because it interleaves test execution with model builds and skips downstream on failure.

Key takeaways

- `dbt test` runs both data tests (generic + singular) and unit tests by default.
- Use `test_type:unit` / `test_type:data` / `test_type:generic` / `test_type:singular` to filter.
- Indirect selection: pick the model, get its tests.
- Tests pass when the underlying query returns 0 rows.
- For production, prefer `dbt build` so failures skip downstream models.

Related

- Data tests, unit tests, custom generic tests, data test configurations.
- `dbt build` — for the integrated build-and-test flow.
- `severity`, `warn_if`, `error_if` configs.

`dbt docs`

Source: https://docs.getdbt.com/reference/commands/cmd-docs **Type:** Documentation · Checkpoint 1

What it does

Generates and serves dbt's documentation website — a browsable view of your project's models, sources, lineage, and column-level metadata.

Two subcommands: `generate` and `serve`. (Fusion 2.0+ replaces `generate` with a `--write-catalog` flag.)

`dbt docs generate`

Three steps under the hood: 1. Copies the docs site's `index.html` into `target/`. 2. Compiles project resources so their `compiled_code` lands in `manifest.json`. 3. Queries the warehouse for metadata → writes `catalog.json` (column names, types, table stats).

```
        dbt docs generate
        dbt docs generate --select +orders     # limit catalog to selected nodes +
upstream
        dbt docs generate --no-compile         # skip step 2
        dbt docs generate --empty-catalog      # skip step 3 (dev shortcut)
        dbt docs generate --static             # single self-contained HTML
```

`--select`

Restricts step 3 (catalog query) to selected nodes. Step 2 (compile) is unaffected.

> **Perf note**: under 100 selected nodes → filter at DB level via `WHERE` (fast). 100+ → query everything in the schemas, then filter in memory. Final `catalog.json` is post-filtered either way.

`--no-compile`

Skip re-compilation. Special macros like `generate_schema_name` still run during parsing.

`--empty-catalog`

Skip the metadata queries. Use only in dev — production docs need column-level info.

`--static`

Bundles `catalog.json` + `manifest.json` into the `index.html` itself, producing a single shareable HTML file (email, S3-served, etc.).

`dbt docs serve`

Starts a local webserver on port 8080 rooted at `target/`, opens the docs in your browser.

```
        dbt docs serve                     # default
        dbt docs serve --port 8001         # custom port
        dbt docs serve --host ""           # all interfaces (Core only; not Cloud
CLI)
        dbt docs serve --no-browser        # don't auto-open
```

Requires `dbt docs generate` to have run first (needs `catalog.json`).

- Available in dbt Core and Cloud CLI; **not** in the dbt Studio IDE.
- v1.8.1+ default host is `127.0.0.1`; earlier versions used `""`.

Fusion: `--write-catalog`

In Fusion 2.0+, you don't run `dbt docs generate` for the catalog — instead pass `--write-catalog` to other commands:

```
        dbt build  --write-catalog
        dbt run    --write-catalog
        dbt parse  --write-catalog
        dbt compile --write-catalog
```

- Hydrates `catalog.json` *during* the run.
- Faster than the separate-command approach.
- Does **not** produce the static docs site HTML — for that, still use `dbt docs generate` with Core.
- dbt Platform jobs on Fusion auto-substitute `--write-catalog` when `dbt docs generate` is called.

Artifacts

- `manifest.json` — your project's parsed/compiled state.
- `catalog.json` — warehouse metadata (columns, types, statistics).
- `index.html` — the docs site shell.

Key takeaways

- `generate` builds the artifacts; `serve` hosts them locally.
- `--select` lets you narrow the catalog query to relevant nodes (fast for <100; in-memory filter beyond).

- `--static` gives you a single-file shareable docs page.
- Fusion uses `--write-catalog` on `build`/`run`/`parse`/`compile` instead of a separate `docs generate`.
- `dbt docs serve` isn't available in Studio IDE.

Related

- `manifest.json` / `catalog.json` reference (artifacts).
- dbt Explorer (Cloud) — Cloud-hosted docs alternative.
- The `description`, `columns`, doc-block conventions that populate the docs site.

`dbt show`

Source: https://docs.getdbt.com/reference/commands/show **Type:** Documentation · Checkpoint 1

What it does

Compile a single resource (or an inline SQL query), execute it against the warehouse, and print a preview of the result rows to the terminal.

What it can preview

- A **model** (single one only)
- A **test** (super useful for inspecting failing-row queries)
- An **analysis**
- An **arbitrary inline query** via `--inline`

Not supported: Python (`dbt-py`) models, multi-node selectors, selector methods, graph operators.

Default behavior

- Prints the first **5 rows**.
- Always compiles and runs the *query* from source — even for a model that's already materialized. (It doesn't `select * from` the existing relation.)

Key flags

`--limit n`

Sets row count. The flag **modifies the SQL itself** (wraps the query in a `LIMIT n` CTE/subquery), so the warehouse only processes and returns `n` rows. This matters for large datasets — it's not just a display limit.

```
dbt show --select stg_orders --limit 20
```

`--inline "..."`

Run ad-hoc SQL. Output goes to logs/terminal, not the warehouse.

```
dbt show --inline "select * from {{ ref('stg_orders') }} where status = 'returned'"
```

⚠️ Since inline SQL is arbitrary, **dbt can't guarantee it won't modify the warehouse**. Use a read-only profile/role to be safe:

```
dbt show --inline "select * from my_table" --profile my-read-only-profile
```

`--output json` (and `--log-format json`)

Machine-readable output. `--output json` switches the rendered preview to JSON; `--log-format json` makes the *entire* log line machine-readable.

```
dbt show --inline "select 1" --output json --log-format json
```

Example response:

```
{"data": {"is_inline": true, "preview": "[{\"ID\": 1}]", ...}}
```

Killer use case: debug a failing test

```
$ dbt build -s "my_model_with_duplicates"
... Failure in test unique_my_model_with_duplicates_id ...

$ dbt show -s "unique_my_model_with_duplicates_id"
| unique_field | n_records |
| ------------ | --------- |
|            1 |         2 |
```

You're directly previewing the test's failing-row SQL — no need to go open the compiled `.sql` file in `target/`.

Common patterns

```
# preview a model
dbt show --select stg_orders

# preview a model with 20 rows
dbt show --select stg_orders --limit 20

# preview a test's failing rows
dbt show --select unique_orders_order_id

# ad-hoc query
dbt show --inline "select count(*) from {{ ref('orders') }}"

# scripting/automation
dbt show --inline "select 1" --output json --log-format json
```

Gotchas

- **One node at a time.** Selector methods and graph operators are ignored.
- **Always re-compiles & re-queries.** Even if the model just ran, `dbt show` doesn't read from the materialized relation.
- **`--inline` is arbitrary SQL.** Use read-only credentials if you don't trust yourself.
- **No Python models.**

Key takeaways

- The fast way to peek at a model, test, or ad-hoc query without opening a SQL editor.
- `--limit` is pushed into the SQL — cheap on big tables.
- `--inline` makes it a SQL REPL; pair with a read-only profile.
- Most powerful when chained with `dbt build` to debug failing tests.

Related

- `dbt test` — generates the failing-row queries that `show` previews.
- `dbt compile` — when you want the SQL but not a result preview.
- Node selection syntax (single-node form).

`dbt snapshot`

Source: https://docs.getdbt.com/reference/commands/snapshot **Type:** Documentation · Checkpoint 1

What it does

Executes the snapshots defined in your project — generating **Type-2 Slowly Changing Dimension** records that track how source data changes over time.

What is a snapshot?

A snapshot captures the state of a row each time the source changes, preserving history that the source itself overwrites destructively. Each row gets `dbt_valid_from`/`dbt_valid_to` timestamps so you can reconstruct point-in-time views.

Where snapshots live

- Defined in **YAML** with a `strategy` (`timestamp` or `check`) and a `unique_key`.
- dbt looks in the directories listed under `snapshot-paths` in `dbt_project.yml`. Default: `snapshots/`.
- You can list multiple paths.

Schedule

Run `dbt snapshot` regularly (typically daily) so changes are captured between runs. If a row changes twice between snapshot runs, you only see the latest version — frequency drives fidelity.

Usage

```
dbt snapshot                                # all snapshots
dbt snapshot --select my_snapshot           # one snapshot
dbt snapshot --select tag:nightly           # by tag
dbt snapshot --exclude my_snapshot
dbt snapshot --help                         # full options
```

Selection

Use `--select` / `--exclude` with standard node selection syntax (graph operators, methods, tags). See *Node selection syntax* for the full grammar.

Global flags

Honors all standard global flags — `--threads`, `--target`, `--profile`, `--profiles-dir`, logging options. See "About flags (global configs)" for the list.

Snapshot vs `dbt build`

- Snapshots can be run via `dbt build` (in DAG order with everything else) — usually what you want in production.
- Use `dbt snapshot` directly when you specifically want to capture history without re-running models/tests/seeds (e.g., a separate "capture state every 15 min" job, with a slower model-build job running hourly).

Output

- Each snapshot becomes a table in the warehouse with the snapshot's name (in the configured snapshot schema/database).
- Added columns: `dbt_valid_from`, `dbt_valid_to` (and a `dbt_scd_id` hash).
- Rows are inserted, never deleted; the open/active version has `dbt_valid_to IS NULL`.

Common operational pattern

A typical setup: - `dbt snapshot` on a 15- or 30-minute cron — fast, lightweight, captures change history. - `dbt build` on an hourly cron — runs models that read from those snapshots.

Gotchas

- Snapshots are stateful — they read the *current* snapshot table to decide what changed. Don't drop or alter them by hand.
- `unique_key` must be truly unique in the source at any given moment; ambiguity causes duplicate history rows.
- The `check` strategy compares specified columns; the `timestamp` strategy uses an `updated_at` column. Pick the one that matches how your source signals changes.

Key takeaways

- `dbt snapshot` runs all snapshots in your project — capturing Type-2 SCD history of source data.
- Defined in YAML with `strategy` + `unique_key`; lives in `snapshots/` by default.
- Run on a schedule appropriate to how fast your source changes.
- Use `dbt build` in prod to integrate with the rest of the DAG; `dbt snapshot` standalone for capture-only jobs.

Related

- Snapshots (build docs) — full reference on `strategy`, `unique_key`, `check_cols`, etc.
- Snapshot configs reference.
- `dbt build` — the integrated runner.

`dbt seed`

Source: https://docs.getdbt.com/reference/commands/seed **Type:** Documentation · Checkpoint 1

What it does

Loads small static CSV files from `seed-paths` (default: `seeds/`) into the warehouse as tables. Use for version-controlled reference data that lives with the project — country codes, region mappings, business categories — not for source data (use an EL tool for that).

How it works

- Each CSV becomes a table named after the file (`country_codes.csv` → `country_codes`).
- Column types are inferred or configurable via seed configs.
- Reference seeds with `{{ ref('country_codes') }}` like any model.

Common commands

```
# All seeds
dbt seed

# One seed
dbt seed --select "country_codes"

# Multiple seeds, with full refresh
dbt seed --select "country_codes state_codes" --full-refresh
```

Selection: standard node selection syntax — paths, tags, graph operators.

Global flags: all dbt globals — `--threads`, `--target`, logging, etc.

`--full-refresh`

Forces a clean reload of the seed (drop + recreate from CSV) rather than an incremental update.

Use when: - The CSV's column names or types changed. - You want consistent state across environments after a seed change. - You just want to be sure you've blown away the old version.

```
dbt seed --full-refresh
dbt seed --select "country_codes" --full-refresh
```

`--empty` (v1.12+)

Creates the seed table with the right schema **but zero rows**. Column names/types are inferred from the CSV.

```
dbt seed --empty
dbt seed --select country_codes --empty
```

Use for: - CI/dev environments where you need the table to exist for downstream models or unit tests, without the cost of loading the data. - Schema-only validation runs that pair with `dbt run --empty` / `dbt build --empty`.

Example output

```
14:46:15 | 1 of 1 START seed file analytics.country_codes...     [RUN]
14:46:15 | 1 of 1 OK loaded seed file analytics.country_codes... [INSERT 3 in
0.01s]
14:46:16 | Finished running 1 seed in 0.14s.
```

Artifacts

Produces a `run_results.json` entry per seed for inspection/troubleshooting.

When NOT to use seeds

- Loading source data from an external system — use an EL tool (Fivetran, Airbyte, custom Python). Seeds aren't a data-ingest pipeline.
- Big files. Seeds are designed for tens of rows up to a few thousand — keep it lookup-table-sized.
- Anything that changes outside source control. Seeds are versioned alongside your project.

Configuring seeds

- Folder path: `seed-paths` in `dbt_project.yml`.
- Column types / quoting / database / schema: under `seeds:` in `dbt_project.yml` or in a seed YAML's `config:` block.
- See *Seed configurations* reference for the full key list.

Common patterns

```
dbt seed                                          # load all seeds
dbt seed --select "country_codes"                 # one seed
dbt seed --full-refresh                           # rebuild all
dbt seed --select my_seed --empty                 # schema-only
dbt build                                         # seeds + models + tests +
snapshots
```

Key takeaways

- Seeds = small CSVs versioned in your project, loaded into the warehouse as tables.
- Reference with `{{ ref('seed_name') }}`.
- `--full-refresh` rebuilds from scratch (use after CSV schema changes).
- `--empty` (v1.12+) creates the table with no rows — for CI/unit tests.
- Don't use seeds for source data — use a proper EL tool.

Related

- Seed configurations (column types, quoting, paths).
- `Add Seeds to your DAG` build docs.
- `dbt build` — integrated runner that includes seeds.

`dbt_project.yml`

Source: https://docs.getdbt.com/reference/dbt_project.yml **Type:** Documentation · Checkpoint 1

What it is

The required project-root file that makes a directory a dbt project. It declares the project name, version, profile, folder paths, and default configurations for every resource type.

How dbt finds it

- Searches current working directory and parents.
- Override via `--project-dir` CLI flag.
- Or `DBT_PROJECT_DIR` env var (renamed to `DBT_ENGINE_PROJECT_DIR` in v1.11+ for Fusion).

Top-level keys (most-used)

```yaml
name: my_project                # required
config-version: 2               # required (current schema)
version: '1.0.0'                # project version
profile: my_profile             # which profile in profiles.yml to use

# Path overrides (all are arrays; default folders shown)
model-paths:    [models]
seed-paths:     [seeds]
test-paths:     [tests]
analysis-paths: [analyses]
macro-paths:    [macros]
snapshot-paths: [snapshots]
docs-paths:     [models, snapshots, analyses, macros]
asset-paths:    [assets]
function-paths: [functions]      # v1.11+ / Fusion

packages-install-path: dbt_packages
clean-targets:        [target, dbt_packages]

require-dbt-version: ">=1.6.0"

query-comment: "/* dbt */"       # injected into every query the warehouse runs

# Cloud-platform binding
dbt-cloud:
  project-id: 123456             # required
  defer-env-id: 67890            # optional
  account-host: cloud.getdbt.com # defaults

# Quoting policy
quoting:
  database: true
  schema:   true
  identifier: true
  # Fusion-only:
  snowflake_ignore_case: true | false

# Hooks
on-run-start: ["{{ create_audit_table() }}"]
on-run-end:   ["GRANT SELECT ON ALL TABLES IN SCHEMA {{ target.schema }} TO reader"]

# Dispatch (override macros from packages)
dispatch:
  - macro_namespace: dbt
    search_order: [my_project, dbt]

# Project-level Mesh setting
restrict-access: true | false

# Resource-type config trees
models:         { ... }
seeds:          { ... }
snapshots:      { ... }
sources:        { ... }
data_tests:     { ... }
exposures:      { ... }
metrics:        { ... }
semantic-models: { ... }
saved-queries: { ... }
analyses:       { ... }
vars:           { ... }
flags:          { ... }
functions:      { ... }
```

The `+` prefix

Inside a resource config tree, prefix configs with `+` (e.g. `+materialized`, `+schema`, `+tags`) to mark them as configs and distinguish from folder-path keys.

```yaml
models:
  my_project:
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
```

Naming convention rule (often missed)

- Inside `dbt_project.yml`: use **dashes** for multi-word resource types (`saved-queries`, `semantic-models`, `model-paths`).
- Inside **other** YAML files (properties, sources): use **underscores** (`saved_queries`, `semantic_models`).

Common patterns

Folder-level materializations & schemas

```yaml
models:
  jaffle_shop:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      finance:
        +schema: finance
      marketing:
        +schema: marketing
```

Package variables

```yaml
vars:
  snowplow:
    'snowplow:timezone': 'America/New_York'
```

Hooks for grants/audits

```yaml
on-run-end:
  - "GRANT SELECT ON ALL TABLES IN SCHEMA {{ target.schema }} TO bi_reader"
```

`flags:` block

Project-level overrides for global configs:

```yaml
flags:
  use_colors: true
  fail_fast: false
```

Properties vs configs

- A **config** is something dbt understands at build time (`materialized`, `tags`, `schema`).
- A **property** is metadata (e.g., macro properties).
- You **cannot** set a property in `dbt_project.yml` — only configs.

Key takeaways

- One file, declares project identity + every resource-type default.
- Dashes inside `dbt_project.yml`; underscores everywhere else.
- The `+` prefix marks configs (vs folder paths) inside resource trees.
- Cascading config: project-wide → folder → model. Most-specific wins.
- Override the project dir via `--project-dir` or env var.

Related

- Project configs reference (each key in detail).
- The `+` prefix and resource paths.
- Global configs / `flags:` block.

dbt Packages

Source: https://docs.getdbt.com/docs/build/packages **Type:** Documentation · Checkpoint 1

What it is

A package is a standalone dbt project (with models, macros, snapshots, etc.) that you import as a library. Adding a package makes its models referencable via `{{ ref('pkg_model') }}` and its macros usable in your own code.

Common use cases

- **Modeled SaaS datasets** — Snowplow events → sessions, Stripe → standardized billing.
- **Macro utilities** — `dbt_utils` (pivots, surrogate keys, schema tests), `audit_helper`.
- **Adapter helpers** — Redshift privileges, Stitch utilities.

How to add a package

1. Create `packages.yml` (or `dependencies.yml`) at project root, next to `dbt_project.yml`.
2. Add packages.
3. `dbt deps` to install. Installs under `dbt_packages/` (default; gitignored).

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.1.1
  - git: "https://github.com/dbt-labs/dbt-utils.git"
    revision: 1.1.1
  - local: /opt/dbt/redshift
```

Sources of packages

Hub (recommended)

```
packages:
  - package: dbt-labs/snowplow
    version: 0.7.3
```

- Requires a version. Pin to a patch range:

```
version: [">=0.7.0", "<0.8.0"]
```

- Only Hub installs let dbt **dedupe shared transitive dependencies** (e.g., both Snowplow and Stripe depend on `dbt_utils`).
- Prereleases: explicit version `0.4.5-a2` or `install_prerelease: true` with a range.

Git

```
packages:
  - git: "https://github.com/dbt-labs/dbt-utils.git"
    revision: 1.1.1      # branch, tag, or 40-char commit
```

Includes `subdirectory:` for packages nested in monorepos.

Internal tarball

```
packages:
  - tarball: https://artifactory.example.com/dbt-utils/1.1.1.tar.gz
    name: 'dbt_utils'
```

Local

```
packages:
  - local: relative/path/to/subdirectory
```

Best for: monorepos, testing in-development packages, nested integration-test projects.

Private packages

Native (recommended)

Uses your existing dbt Cloud git integration — no token wrangling. Supports GitHub, GitLab, Azure DevOps.

```
packages:
  - private: dbt-labs/awesome_repo
    revision: "0.9.5"
    provider: github       # required for Fusion or multi-integration setups
```

Fusion locally uses your SSH config.

SSH key (CLI only)

```
packages:
  - git: "git@github.com:dbt-labs/dbt-utils.git"
```

Not supported on dbt Cloud.

Git token (HTTPS)

```
packages:
  - git: "https://{{env_var('DBT_ENV_SECRET_GIT_CREDENTIAL')}}@github.com/dbt-labs/awesome_repo.git"
```

dbt Cloud env vars must be prefixed `DBT_` or `DBT_ENV_SECRET_`.

`dbt deps` and pinning

- `dbt deps` installs/updates packages.
- Creates/updates `package-lock.yml` (commit this!) — pins every install to a specific commit/version.
- Subsequent `dbt deps` runs reuse the lock until `packages.yml` changes.
- To force upgrades: `dbt deps --upgrade`.
- Unpinned git packages produce a warning; silence with `warn-unpinned: false` (not recommended).

Configuring an installed package

You can configure a package's models, seeds, and vars from your `dbt_project.yml`:

```
vars:
  snowplow:
    'snowplow:timezone': 'America/New_York'

models:
  snowplow:
    +schema: snowplow

seeds:
  snowplow:
    +schema: snowplow_seeds
```

Configs in *your* `dbt_project.yml` override anything in the package's own config.

Updating and uninstalling

- **Update**: change version in `packages.yml` → `dbt deps`. You may also need `dbt run --full-refresh` for affected models.
- **Uninstall**: remove from `packages.yml`. Either delete the package folder under `dbt_packages/` or run `dbt clean` to nuke everything, then `dbt deps`.

Key takeaways

- Packages = importable dbt projects; reference their models via `ref`.
- Install via Hub (preferred — handles transitive deps), git, tarball, local, or private.
- Always pin (or use a range) — commit `package-lock.yml`.
- Configure package models from your own `dbt_project.yml`; your configs win.
- Private packages: prefer the `private:` key with `provider:`.

Related

- `dbt deps` command.
- `dbt clean`.
- `dispatch` config — override package macros.
- Mesh / `private:` packages for governance.

Python Models

Source: https://docs.getdbt.com/docs/build/python-models **Type:** Documentation · Checkpoint 1

What it is

A dbt Python (`dbt-py`) model is a `.py` file in `models/` whose `model()` function returns a DataFrame. dbt persists that DataFrame as a table in your warehouse. Python models join the same DAG as SQL models — same `ref`, same tests, same documentation, same lineage.

Supported platforms

- Snowflake (Snowpark)
- BigQuery
- Databricks (PySpark)

Python models for Snowflake/BigQuery/Databricks are also supported in Fusion.

Anatomy of a Python model

```
# models/my_python_model.py
def model(dbt, session):
    upstream_df = dbt.ref("stg_orders")
    upstream_src = dbt.source("jaffle_shop", "customers")

    # transformations you couldn't do in SQL
    final_df = ...

    return final_df
```

Two required arguments: - `dbt` — a class dbt builds for each model. Provides `dbt.ref()`, `dbt.source()`, `dbt.this()`, `dbt.is_incremental`, `dbt.config()`, `dbt.config.get()`, `dbt.config.meta_get()`. - `session` — the platform's connection (Snowpark, BigFrames, SparkSession). dbt passes it in explicitly.

Required return: a single DataFrame. - Snowflake: Snowpark or pandas - BigQuery: BigFrames, pandas, or Spark - Databricks: Spark, pandas, or pandas-on-Spark

Materializations

Only **table** (default) and **incremental**. No view, no ephemeral. (Ephemeral refs aren't supported either.)

```
def model(dbt, session):
    dbt.config(materialized="incremental")
    df = dbt.ref("upstream_table")

    if dbt.is_incremental:
        max_from_this = f"select max(updated_at) from {dbt.this}"
        df = df.filter(df.updated_at >= session.sql(max_from_this).collect()[0][0])

    return df
```

Incremental strategies are adapter-dependent. BigQuery/Dataproc: `merge` works, `insert_overwrite` does not.

Configuration

Three options

1. `dbt_project.yml` (folder-wide).
2. A `.yml` properties file.
3. `dbt.config()` inside the model.

`dbt.config()` only accepts **literal** values — strings, bools, numbers — plus dynamic config access. No functions, no nested data. dbt parses it statically.

YAML config with column tests + custom tests

```yaml
models:
  - name: my_python_model
    description: My Python transformation
    config:
      materialized: table
      tags: ['python']
    columns:
      - name: id
        data_tests:
          - unique
          - not_null
    data_tests:
      - custom_generic_test
```

Accessing vars, env_vars, target

Set them through YAML config (Jinja-rendered there), then read via `dbt.config.get()`:

```yaml
models:
  - name: my_python_model
    config:
      target_name: "{{ target.name }}"
      specific_var: "{{ var('SPECIFIC_VAR') }}"
```

```python
def model(dbt, session):
    if dbt.config.get("target_name") == "dev":
        df = dbt.ref("fct_orders").limit(500)
```

Accessing meta

```yaml
config:
  meta:
    custom_value: "111"
```

```python
value = dbt.config.meta_get("custom_value")
# or
value = dbt.config.get("meta", {}).get("custom_value")
```

Dynamic config (v1.8+)

```python
print(f"my_var = {dbt.config.get('my_var')}")
```

What you get from `dbt`

- `dbt.ref("model")` → upstream DataFrame.
- `dbt.source("src","tbl")` → source DataFrame.
- `dbt.this`, `dbt.this.database`, `.schema`, `.identifier` → current model location.
- `dbt.is_incremental` → boolean (true on incremental rebuilds, not initial).
- `dbt.config(...)`, `dbt.config.get(...)`, `dbt.config.meta_get(...)`.

Execution model

Code runs **remotely on the data platform**, not on the dbt host. dbt separates definition from execution. You declare *what*; the warehouse runs *how*.

Limits / gotchas

- No Jinja in `.py` files — context comes from the `dbt` argument.
- Can't `ref()` ephemeral models.
- `dbt.config()` is static — no dynamic Python at config time.
- `dbt show` does not support Python models.
- Adapter-specific incremental strategies; check per-platform docs.

Referencing Python from SQL

Totally fine:

```sql
-- models/downstream.sql
with up as (
    select * from {{ ref('my_python_model') }}
)
...
```

Key takeaways

- Python models = `.py` files defining `model(dbt, session)` that return a DataFrame.
- Supported on Snowflake, BigQuery, Databricks (and Fusion).
- Materializations: only `table` and `incremental`.
- `dbt.ref()` / `dbt.source()` instead of Jinja; `dbt.config.get()` for vars/env/target.
- Tests, docs, lineage, selection — all the same as SQL models.

Related

- Materializations doc — for the table/incremental limits.
- Incremental strategy — for adapter-specific options.
- Snapshots / tests — not supported in Python.

`grants` Resource Config

Source: https://docs.getdbt.com/reference/resource-configs/grants **Type:** Documentation · Checkpoint 1

What it is

A resource config that lets you define warehouse `GRANT` statements declaratively in your dbt project. When the model/seed/snapshot is built, dbt issues `GRANT`/`REVOKE` so the object's permissions exactly match your config.

Applies to: **models, seeds, snapshots** (not sources, not tests).

Two pieces

- **Privilege** — the action being granted: `select`, `insert`, etc.
- **Grantees** — the recipients (users, groups, roles on Snowflake, service accounts on BigQuery).

Where to configure

In `dbt_project.yml`

```yaml
models:
  +grants:
    select: ['user_a', 'user_b']
```

In a properties YAML

```yaml
models:
  - name: specific_model
    config:
      grants:
        select: ['reporter', 'bi']
```

In the model SQL via `config()`

```sql
{{ config(grants = {'select': ['user_c']}) }}
```

Same `grants:` syntax for seeds and snapshots — under `seeds:` or `snapshots:` keys.

Inheritance: clobber vs add

Default = clobber

The more-specific config replaces the less-specific list.

```yaml
# dbt_project.yml
models:
  +grants:
    select: ['user_a', 'user_b']
```

```sql
{{ config(grants = {'select': ['user_c']}) }}
```

Result: `specific_model` grants `select` to **user_c only**.

Add with `+` prefix

Prefix the privilege name with `+` to **append** rather than replace:

```sql
{{ config(grants = {'+select': ['user_c']}) }}
```

Result: `user_a`, `user_b`, AND `user_c` all get `select`.

Notes: - The `+` controls merge behavior **per privilege**. Privileges without `+` still clobber. - This `+` (inside the grants dict) is distinct from the `+` prefix used to mark configs in `dbt_project.yml`. - `grants` is currently the only config supporting `+`-prefixed merge behavior.

Conditional grants (Jinja)

```yaml
models:
  +grants:
    select: "{{ ['user_a', 'user_b'] if target.name == 'prod' else ['user_c'] }}"
```

Revoking

dbt only manages grants where a `grants` config is attached. Three behaviors:

To…	Do this
Revoke from one user	Remove them from the grants list
Revoke from everyone	Set the list to `[]` (empty)
Stop managing grants	Delete the `+grants:` block entirely — dbt leaves existing grants alone

```yaml
models:
  +grants:
    select: []          # revokes all grantees of 'select'
```

When to fall back to hooks

`grants` is the right call for typical view/table SELECT/INSERT permissions. Use `on-run-end` or `post-hook` SQL when you need: - Permissions on database objects

other than views/tables (e.g., functions, schemas). - Row-level or column-level security, masking policies, **future grants**. - Platform-specific advanced features not exposed by the config. - Complex custom logic.

Efficiency

dbt picks the most efficient pattern per adapter — replacing object vs. updating in place. Inspect debug logs to see the exact GRANT/REVOKE statements emitted.

Common patterns

```
        # Default for the whole project
        models:
          +grants:
            select: ['bi_reader']

        # More permissive for marts
        models:
          my_project:
            marts:
              +grants:
                +select: ['analyst', 'data_science']    # adds to inherited bi_reader
```

```
        # Different grants in prod vs dev
        models:
          +grants:
            select: "{{ ['bi_prod'] if target.name == 'prod' else ['bi_dev'] }}"
```

Key takeaways

- Define grants as configs in `dbt_project.yml`, YAML, or model SQL.
- Default merge = clobber; use `+select`/`+insert`/etc. to add.
- Empty list `[]` revokes all; deleted block stops management.
- Use Jinja for env-specific grants.
- Drop to hooks only when grants config can't express what you need.

Related

- Hooks & operations — when grants aren't enough.
- `dbt_project.yml` `+` prefix (config marker) vs `+` inside grants dict (merge marker).
- Resource configs reference.

Snapshots — Add Snapshots to Your DAG

Source: https://docs.getdbt.com/docs/build/snapshots **Type:** Documentation · Checkpoint 1

What it is

Snapshots implement **Type-2 Slowly Changing Dimensions** on top of mutable source tables. They record changes over time by adding rows with validity windows (`dbt_valid_from` / `dbt_valid_to`).

The problem they solve

Source data: | id | status | updated_at | |—-|———|———-| | 1 | pending | 2024-01-01 |

After the order ships, source overwrites the row: | id | status | updated_at | |—-|———|———-| | 1 | shipped | 2024-01-02 |

You've lost the "pending" history. Snapshot table: | id | status | updated_at | dbt_valid_from | dbt_valid_to | |—-|———|———-|———————-|———-| | 1 | pending | 2024-01-01| 2024-01-01 | 2024-01-02 | | 1 | shipped | 2024-01-02| 2024-01-02 | NULL |

Configuration (v1.9+ YAML form)

```
        # snapshots/orders_snapshot.yml
        snapshots:
          - name: orders_snapshot
            relation: source('jaffle_shop', 'orders')
            config:
              schema: snapshots
              unique_key: id
              strategy: timestamp
              updated_at: updated_at
              dbt_valid_to_current: "to_date('9999-12-31')"   # optional
              hard_deletes: invalidate                          # ignore | invalidate
| new_record
```

Config	Required?	Notes
`strategy`	✅	`timestamp` or `check`
`unique_key`	✅	column name, array, or expression
`updated_at`	for `timestamp` strategy	column tracking last-modified time
`check_cols`	for `check` strategy	list of columns or `'all'`
`database` / `schema` / `alias`	optional	(v1.9 made `target_schema`/`target_database` optional)
`dbt_valid_to_current`	optional	replace NULL with a sentinel value (e.g. `9999-12-31`)
`snapshot_meta_column_names`	optional	rename `dbt_valid_from` etc.
`hard_deletes`	optional	how to handle deletes — see below

Two strategies

`timestamp` (recommended)

Uses an `updated_at` column. If a row's `updated_at` > the snapshot's last seen value, the old version is invalidated and a new row inserted. - Tracks **one** column. - Handles new/removed source columns gracefully. - Doesn't need updating when source schema evolves.

`check`

Compares specified columns between current and historical values. Use when no reliable `updated_at` exists.

```
        strategy: check
        check_cols: ["status", "shipping_address"]
        # or: check_cols: 'all'   (not recommended — fragile to schema change)
```

Hard deletes (v1.9+)

- `ignore` (default) — deleted source rows just stop updating; their last version stays "current".
- `invalidate` — closes the validity window (sets `dbt_valid_to`).
- `new_record` — inserts a new row marking the deletion.

Add a snapshot — workflow

1. Create `snapshots/orders_snapshot.yml`.
2. Pick `timestamp` if you have a reliable `updated_at`; otherwise `check`.
3. (Optional) Use an ephemeral model to clean/dedup source before snapshotting:

```
        -- models/ephemeral_orders.sql
        {{ config(materialized='ephemeral') }}
        select * from {{ source('jaffle_shop','orders') }}
```

   Then `relation: ref('ephemeral_orders')`.
4. `dbt snapshot` — initial run captures current state with `dbt_valid_to = NULL` (or `dbt_valid_to_current`).
5. Reference downstream: `{{ ref('orders_snapshot') }}`.
6. Schedule `dbt snapshot` regularly.

How it works

- **First run:** select-statement output + meta columns; all rows current.
- **Subsequent runs:** changed rows get `dbt_valid_to` set, new versions inserted with `dbt_valid_to = NULL`.

Best practices

- ✅ Prefer `timestamp` strategy.
- ✅ Use `dbt_valid_to_current` for easier range queries.
- ✅ Add a uniqueness test on the source for `unique_key` — duplicate keys destroy history.
- ✅ Keep snapshots in a **separate schema**, restricted permissions. Snapshots cannot be rebuilt.
- ✅ Use an ephemeral model to clean source before snapshotting.

v1.12+ tip

You can `dbt compile` to inspect the SQL dbt generates for the snapshot — the file lands in `target/compiled/`.

Key takeaways

- Snapshots = T2 SCD: keep `dbt_valid_from`/`dbt_valid_to` for every change.
- `timestamp` strategy unless you can't.
- Snapshots are write-only history — protect them; don't drop.
- v1.9 made YAML the canonical config format and added `hard_deletes`.
- Define `unique_key` truly uniquely; test it.

Related

- `dbt snapshot` command.
- Snapshot configs reference.
- Snapshot meta columns / `snapshot_meta_column_names`.

About Incremental Models

Source: https://docs.getdbt.com/docs/build/incremental-models-overview **Type:** Documentation · Checkpoint 1

What it is

A materialization that updates a warehouse table by transforming and loading **only new or changed rows** since the last run. Instead of rebuilding the whole table every run, append or update only the delta.

When to use

Reach for incremental only when you have a real problem with table builds: - Source tables with millions/billions of rows. - Slow/complex transformations (regex, UDFs, heavy joins) where a full rebuild costs significant time or money.

It's not a default. Quoting the docs verbatim: *"We recommend using them when your dbt runs are becoming too slow."* For everything else, prefer `view` then `table`.

The standard escalation

1. **view** — cheap, always fresh.
2. **table** — fast queries, full rebuild each run.
3. **incremental** — fast queries, partial rebuild each run.

Only escalate when the prior tier's build time becomes intolerable.

How it works under the hood

- **Adapters that support MERGE**: dbt emits a `MERGE` to upsert new and changed rows.
- **Adapters without MERGE**: dbt does `DELETE` of matching keys, then `INSERT`.
- **Transaction management**: where supported, dbt runs the operation in a transaction; on failure, the warehouse rolls back so the table stays in a consistent state.

Configuration shape (minimum)

```
{{ config(
    materialized='incremental',
    unique_key='id'
) }}

select *
from {{ ref('events') }}

{% if is_incremental() %}
where event_time > (select max(event_time) from {{ this }})
{% endif %}
```

Two parts: 1. **Config** declares incremental + a `unique_key` (used by the strategy). 2. **A guarded WHERE clause** filters source to "only new rows" on incremental runs. On a first run / `--full-refresh`, the WHERE is skipped and the table is built from scratch.

What "new" means

You decide. The `is_incremental()` guard runs only when the model already exists and `--full-refresh` isn't set. Inside the guard, compare against `{{ this }}` (the existing table) to pull only rows newer than the last run.

Trade-offs

Pros - Big drop in build time and warehouse cost on large tables. - Pairs well with append-mostly event data.

Cons - Extra configuration burden. - Wrong `unique_key` → duplicate rows. - Late-arriving data may be missed if your filter is too tight (see `lookback` / microbatch). - Schema changes require `--full-refresh`. - Debugging is harder; the model only ever shows you a slice each run.

Related strategies (separate docs)

- **`merge`** / **`append`** / **`delete+insert`** / **`insert_overwrite`** — the strategy options selecting how dbt actually writes the new rows. See *Incremental strategy*.
- **`microbatch`** — newer strategy for large time-series datasets; processes data in independent, idempotent time batches.

Limitations & caveats

- Late-arriving data is the classic incremental pitfall. Either widen your `WHERE` (incurring re-scans), or use microbatch with `lookback`.
- "Limits of incrementality" (a Tristan Handy classic) — schema drift, unique-key drift, and incomplete data can silently corrupt incremental output. Test the model's primary key for uniqueness.
- `--full-refresh` rebuilds from scratch — keep it in your runbook.

Key takeaways

- Incremental = partial rebuild based on a `unique_key` and a `is_incremental()`-guarded filter.
- Don't reach for it until you've felt the pain of full-refresh tables.
- The actual SQL behavior depends on `incremental_strategy` and your adapter's support.
- Microbatch is the modern answer for time-series; pre-existing strategies are still fine elsewhere.
- Use `dbt run --full-refresh` when schema or logic changes.

Related

- Incremental strategy (which one to pick, per adapter).
- Incremental microbatch (the time-series specialization).
- Materializations best practices (the view→table→incremental ladder).

Incremental Strategy

Source: https://docs.getdbt.com/docs/build/incremental-strategy **Type:** Documentation · Checkpoint 1

What it is

`incremental_strategy` configures *how* dbt writes new/changed rows in an incremental model. The right choice depends on volume, unique-key reliability, and adapter support.

Strategy support matrix (paraphrased)

Adapter	append	merge	delete+insert	insert_overwrite	microbatch
postgres	✅	✅	✅	–	✅
redshift	✅	✅	✅	–	✅
bigquery	–	✅	–	✅	✅
spark	✅	✅	–	✅	✅
databricks	✅	✅	✅	✅	✅
snowflake	✅	✅	✅	✅	✅
trino	✅	✅	✅	–	✅
fabric	✅	✅	✅	–	–
athena	✅	✅	–	✅	✅

Configuring

```
# dbt_project.yml — project-wide default
models:
  +incremental_strategy: insert_overwrite
```

```
-- per-model
{{ config(
    materialized='incremental',
    unique_key='date_day',
    incremental_strategy='delete+insert'
) }}
select ...
```

Built-in strategies

`append`

Just `INSERT` new rows. No upserts, no dedup. - ✅ Simple, cheap. - ❌ Duplicates if a row repeats. Not SCD1, only loosely SCD2-ish (adds rows but no versioning).

`merge`

`MERGE` — insert new, update existing — matched by `unique_key`. Conceptually SCD1 (overwrite). - Configurable: `merge_update_columns` (only update these), `merge_exclude_columns` (update all except these). - Default = overwrite *all* columns of matched rows.

```
{{ config(
    materialized = 'incremental',
    unique_key = 'id',
    merge_update_columns = ['email', 'ip_address']
) }}
```

or:

```
{{ config(
    merge_exclude_columns = ['created_at']
) }}
```

- In `incremental_predicates`, alias columns with `DBT_INTERNAL_DEST` (existing) and `DBT_INTERNAL_SOURCE` (new) when disambiguation is needed.

`delete+insert`

`DELETE` matching `unique_key`, then `INSERT`. Useful when `merge` isn't supported or the key isn't truly unique. - Less efficient on large tables than `merge`. - Not SCD-anything — overwrites without history. - ⚠️ For true SCD2, use **snapshots**, not this.

`insert_overwrite`

Replace entire partitions identified by `partition_by`. Most efficient strategy on partition-aware adapters (BigQuery, Spark, Databricks, Snowflake) for time-partitioned data. - Requires partitions (e.g., a date column). - Works well with predictable backfill granularities.

`microbatch`

Specialized strategy for large time-series. Splits each run into independent, idempotent batches over an `event_time` column. (Full coverage in its own doc.)

`incremental_predicates`

Additional filter conditions dbt applies during MERGE / DELETE+INSERT. Reduces the rows the warehouse compares — big perf win on large tables.

Custom strategies

Define a macro named `get_incremental_<strategy>_sql(arg_dict)` and reference it by `incremental_strategy='<strategy>'`. The built-ins have these names:

Strategy	Macro
`append`	`get_incremental_append_sql`
`delete+insert`	`get_incremental_delete_insert_sql`
`merge`	`get_incremental_merge_sql`
`insert_overwrite`	`get_incremental_insert_overwrite_sql`
`microbatch`	`get_incremental_microbatch_sql`

Picking one (quick guide)

- **Append-only event log, can tolerate dupes, want speed**: `append`.
- **Need upsert + unique key**: `merge` (or `delete+insert` if merge unsupported/key not unique).
- **Time-partitioned table**: `insert_overwrite` (partition-aware adapters).
- **Large time-series with backfill / late data**: `microbatch`.
- **True SCD2 history**: not a strategy — use a **snapshot**.

Key takeaways

- `incremental_strategy` controls the write mechanism for incremental models.
- Five built-ins: `append`, `merge`, `delete+insert`, `insert_overwrite`, `microbatch`.
- Adapter support varies; check the matrix.
- `merge_update_columns` / `merge_exclude_columns` give fine-grained MERGE control.
- For SCD2 → snapshots, not `delete+insert`.

Related

- Incremental microbatch (its own doc).
- Snapshots — for SCD2.
- Adapter resource configs — per-platform specifics.

Microbatch Incremental Models

Source: https://docs.getdbt.com/docs/build/incremental-microbatch **Type:** Documentation · Checkpoint 1

What it is

An incremental strategy purpose-built for **large time-series datasets**. Splits each run into independent, idempotent batches defined by an `event_time` column. Available in dbt 1.9+ / Latest release track.

Why it exists

Traditional incremental strategies operate on “all new data” in one query. Microbatch lets dbt: - Process new data in small, time-bounded queries. - Reprocess failed batches independently (`dbt retry`). - Run batches in parallel (auto-detected or `concurrent_batches: true`). - Do backfills without writing custom conditional Jinja.

How a microbatch model works

- You declare `event_time`, `begin`, and `batch_size`.
- dbt splits the run into one query per batch (default `day`).
- Each batch query filters refs/sources to that batch’s time window.
- dbt then inserts/replaces the batch atomically using the most efficient adapter-specific mechanism.

Per-adapter mechanism

Adapter	Underlying op
postgres	`merge`
redshift	`delete+insert`
snowflake	`delete+insert`
bigquery	`insert_overwrite`
spark	`insert_overwrite`
databricks	`replace_where`

Required config

```
{{ config(
    materialized='incremental',
    incremental_strategy='microbatch',
    event_time='session_start',
    begin='2020-01-01',
    batch_size='day'   -- hour | day | month | year
) }}
```

Config	Required?	Default	Notes
`event_time`	✅	–	column on the model AND on direct parents to be filtered
`begin`	✅	–	first batch’s start date (used for initial / full-refresh)
`batch_size`	✅	–	`hour`, `day`, `month`, `year`
`lookback`	optional	`1`	reprocess N prior batches to catch late-arriving data
`concurrent_batches`	optional	auto	force parallel/ sequential execution

Adapter-specific extras

- `dbt-postgres`: `unique_key` required (uses merge).
- `dbt-spark`, `dbt-bigquery`: `partition_by` required.

You must also set `event_time` on upstream models

For dbt to filter a ref to a batch’s window, that upstream must declare an `event_time` too. Dimensional/static parents (e.g. `customers`) can be left unset — they’ll do a full scan in each batch.

```
models:
  - name: page_views
    config:
      event_time: page_view_start
```

Worked example

```
-- models/sessions.sql
{{ config(
    materialized='incremental',
    incremental_strategy='microbatch',
    event_time='session_start',
    begin='2020-01-01',
    batch_size='day'
) }}

with page_views as ( select * from {{ ref('page_views') }} ),  -- auto-filtered
     customers  as ( select * from {{ ref('customers') }} )      -- not filtered

select
  page_views.id as session_id,
  page_views.page_view_start as session_start,
  customers.*
from page_views
left join customers on page_views.customer_id = customers.id
```

For an Oct 1 batch, dbt compiles `where page_view_start >= '2024-10-01' and < '2024-10-02'`. For Oct 2, the next bounded window — totally independent of Oct 1.

Idempotency

Each batch is atomic and idempotent: same input data → same output table for that batch, no matter how many times you run it.

Full-refresh behavior

- Best practice: set `full_refresh: false` on microbatch models — they’re designed to backfill from `begin` instead.
- `dbt run --full-refresh` alone won’t reset data unless `begin` is set.

Backfills

Run a model over an arbitrary range with explicit time bounds (CLI flags / `--event-time-start` and `--event-time-end`) without writing conditional Jinja.

Retry

`dbt retry` reruns only the failed batches — not the entire model. Each batch’s success/failure is tracked independently.

When NOT to use microbatch

- No reliable `event_time` column.
- You need custom merge logic across non-time keys.
- Updates are random in time (not append-mostly).

Key takeaways

- Microbatch = time-windowed, idempotent incremental for big time-series.
- Required: `event_time`, `begin`, `batch_size`.
- Set `event_time` on parents you want filtered; static parents stay un-filtered.
- Adapter-specific extras: `unique_key` (postgres), `partition_by` (spark/bigquery).
- Failed batches retry independently; backfills are first-class.
- Pair with `full_refresh: false` so you don’t accidentally blow the table away.

Related

- `event_time` resource config.
- `dbt retry`.
- Parallel batch execution (`concurrent_batches`).
- Incremental strategy (the parent doc).

The `--empty` Flag

Source: https://docs.getdbt.com/docs/build/empty-flag **Type:** Documentation · Checkpoint 1

What it is

A CLI flag that runs your models against the warehouse but **limits refs and sources to zero rows** — so SQL still compiles and executes, but no input data is scanned. Validates semantic correctness and dependency wiring cheaply.

Where it's supported

- `dbt run`
- `dbt build`
- `dbt snapshot`
- `dbt compile`
- `dbt seed` (from dbt Core v1.12+)

❌ **Not supported on Python models.** The flag is silently ignored.

What it does (mechanically)

- Wraps each `ref` and `source` in a CTE that limits to 0 rows.
- The model's SQL still hits the warehouse.
- Schema gets created; tables/views are built with the right column names and types but **no data**.

Why use it

- **CI / PR validation**: prove that your models will compile and build against the real warehouse without paying to scan/load source data.
- **Dev iteration**: faster than a full run when you just want to check that the model wires up correctly.
- **Schema dry runs**: ensure the table structure exists in dev/CI for downstream unit tests.

Examples

```
# Build the entire project's schemas with empty tables
dbt run --empty

# Empty-build just one model
dbt run --select path/to/your_model --empty

# Compile only (no warehouse execution at all)
dbt compile --empty

# Schema-only seeds (v1.12+) — create the seed table but load no rows
dbt seed --empty

# Combined with dbt build (also includes tests, etc.)
dbt build --empty
```

How it pairs with other things

- `dbt seed --empty` (v1.12+) lets downstream models that depend on seeds compile/run with the seed table existing-but-empty.
- Useful in CI: run `dbt build --empty` to ensure new SQL compiles end-to-end without spending warehouse credits.
- Different from `--sample`, which gives you a *small slice* of real data instead of zero rows.

What it doesn't do

- It does not skip the warehouse round-trip — SQL is still sent and executed (just against empty inputs).
- It doesn't validate **runtime** correctness on real data; that requires a regular run or sample-mode run.
- It doesn't skip tests by itself — pair with selection if you want to limit scope.

Key takeaways

- `--empty` = "build the schema, skip the data."
- Supported on `run`, `build`, `snapshot`, `compile`, and (v1.12+) `seed`.
- Not supported for Python models.
- Cheap CI validation — proves models compile + build without scanning input data.
- Pair with `dbt seed --empty` so downstream-of-seed models can be validated too.

Related

- `--sample` flag — for time-bounded data sampling.
- `dbt compile` — purely compile, no warehouse hits.
- Slim CI patterns / `state:modified+ --defer`.

The `--sample` Flag

Source: https://docs.getdbt.com/docs/build/sample-flag **Type:** Documentation · Checkpoint 1

What it is

A CLI flag that runs models against a **time-bounded slice** of real data — a step beyond `--empty` (which uses zero rows). Builds tables with a sampling of real records, so you can validate transformations without paying to scan full datasets.

Where it's supported

- `dbt run`
- `dbt build`

❌ **Not supported on Python models** (silently ignored).

ℹ️ **Seeds** are created normally, but **referenced** seeds get sampled when downstream models read them.

How it works

Sample mode generates **filtered** refs and sources, filtered by the model's `event_time` column. For this to work: - Every `ref()` you want sampled must point to a model that declares `event_time`. - Sources can also declare `event_time` for the same purpose.

Two sample-spec formats

Relative time spec

Filter from "now" backwards by a granularity: - `hours`, `days`, `months`, `years`

```
dbt run --select path/to/stg_customers --sample="3 days"
dbt run --select path/to/stg_orders    --sample="6 hours"
```

Static time spec

Filter between two explicit boundaries (date or timestamp):

```
dbt run --sample="{'start': '2024-07-01', 'end': '2024-07-08 18:00:00'}"
```

Use this when you want to replay a specific historical window — your busiest week, a known-bad day, etc.

Opting a single ref out of sampling

Append `.render()` to a `ref()` to bypass sampling for that one reference:

```
with source as (
    select * from {{ ref('stg_customers').render() }}    -- unfiltered
)
```

Common for dimensional joins where you need the full lookup table even when fact tables are sampled.

What sample mode is good at

- Faster dev/CI cycles than full builds.
- Validating logic against representative real data.
- Reducing warehouse spend during development.

What it can't fix

- Joins where the sample window misses matching rows on one side — you can end up with incomplete or empty joins.
- Calculations that need full historical context (window functions over all time, lifetime values, etc.).
- Non-time-based data filtering — currently only **time-based sampling** is supported.

Examples

```
# Last 3 days of data
dbt run --select stg_customers --sample="3 days"

# Specific historical window for the whole project
dbt run --sample="{'start': '2024-07-01', 'end': '2024-07-08'}"

# Build (includes tests) with a 6-hour sample
dbt build --sample="6 hours"
```

`--sample` vs `--empty` vs full run

Flag	Input data	Use case
`--empty`	0 rows	Schema/compile validation; CI
`--sample`	Time-bounded slice	Realistic dev iteration with cheap reads
(none)	All rows	Production builds

Requirements checklist

- Models being sampled must have `event_time` configured.
- Any upstream ref you want filtered must also declare `event_time`.

- Use `.render()` to bypass on individual refs.

Key takeaways

- `--sample` builds models with a **time-bounded subset** of real data.
- Supported on `run` and `build`; not on Python models.
- Requires `event_time` on sampled models and their parents.
- Relative (`"3 days"`) or static (`{'start':..., 'end':...}`) specs.

- `.render()` bypasses sampling for one ref.

Related

- `--empty` flag — zero-row companion for cheaper compile validation.
- `event_time` resource config.
- Microbatch incremental strategy (same `event_time` plumbing).